

FPP3 Chapter 9 ARIMA Exercises (홀수 문제)

2176071 김수민

2026-05-12

```
library(fpp3)
library(tsibble)
library(dplyr)
library(ggplot2)
library(gridExtra)
```

Exercise 1

Figure 9.32에는 36개, 360개, 1000개 랜덤 숫자의 ACF가 나와 있다.

(a) 각 그래프의 차이 설명 및 백색잡음 여부

세 ACF 그래프 모두 백색잡음(White Noise)을 나타낸다.

다만 표본 크기(n)에 따라 임계값 $\pm \frac{1.96}{\sqrt{n}}$ 이 달라지므로, 그래프의 모양에 차이가 생긴다.

- $n = 36$: 임계값이 가장 넓다 ($\pm 1.96/\sqrt{36} \approx \pm 0.327$).
표본이 작아 우연에 의한 자기상관이 비교적 크게 나타날 수 있다.
- $n = 360$: 임계값이 좁아진다 ($\pm 1.96/\sqrt{360} \approx \pm 0.103$).
자기상관값들이 더 0에 가깝게 분포한다.
- $n = 1000$: 임계값이 가장 좁다 ($\pm 1.96/\sqrt{1000} \approx \pm 0.062$).
표본이 클수록 각 추정된 자기상관값이 0에 더 정밀하게 수렴한다.

세 경우 모두 임계값 내에 자기상관값들이 위치하므로, 모두 백색잡음이라고 판단한다.

(b) 임계값이 다른 이유 / 자기상관이 다른 이유

임계값이 다른 이유: 표본자기상관계수의 표준오차는 $\approx 1/\sqrt{n}$ 이므로,
표본이 클수록 임계값이 작아진다 (더 정밀한 추정).

자기상관이 다른 이유: 자기상관 추정값 $\hat{r}_k$ 는 표본에 따라 달라지는 확률변수이다.
각 그래프는 서로 다른 난수 집합에서 생성되었기 때문에,
같은 백색잡음 과정이라도 추정된 자기상관값들이 서로 다르게 나타난다.
표본이 클수록 분산이 줄어 0에 더 가까워지는 경향이 있다.

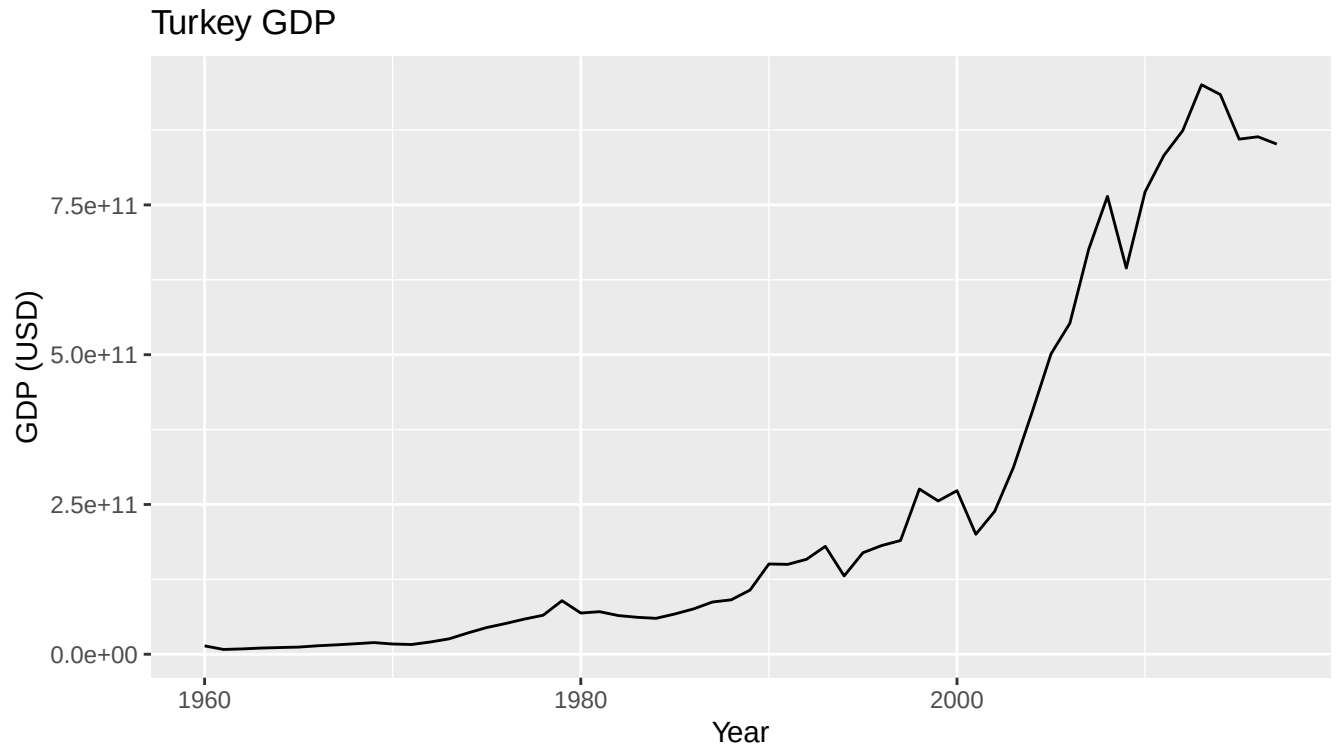
Exercise 3

다음 시계열에 대해 적절한 Box-Cox 변환과 차분 차수를 구하여 정상성을 확보하라.

(a) Turkish GDP from `global_economy`

```
turkey_gdp <- global_economy |>
  filter(Country == "Turkey")

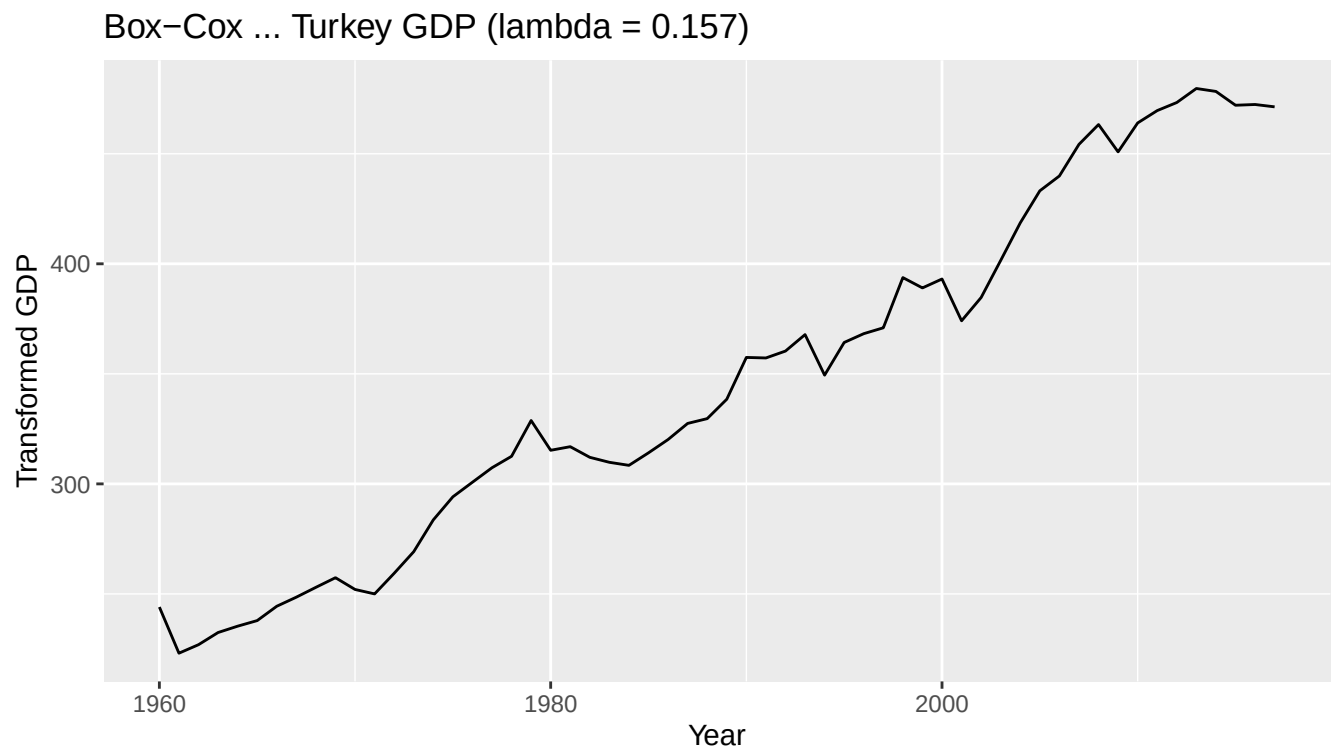
#
turkey_gdp |>
  autoplot(GDP) +
  labs(title = "Turkey GDP", y = "GDP (USD)", x = "Year")
```



```
# Box-Cox
lambda_turkey <- turkey_gdp |>
  features(GDP, features = guerrero) |>
  pull(lambda_guerrero)
cat("Guerrero lambda:", round(lambda_turkey, 3), "\n")
```

```
## Guerrero lambda: 0.157
```

```
#
turkey_gdp |>
  mutate(GDP_bc = box_cox(GDP, lambda_turkey)) |>
  autoplot(GDP_bc) +
  labs(
    title = paste0(
      "Box-Cox   Turkey GDP (lambda = ",
      round(lambda_turkey, 3), ")"),
    y = "Transformed GDP", x = "Year"
  )
```



```
#
turkey_gdp |>
  mutate(GDP_bc = box_cox(GDP, lambda_turkey)) |>
  features(GDP_bc, unitroot_ndiffs)
```

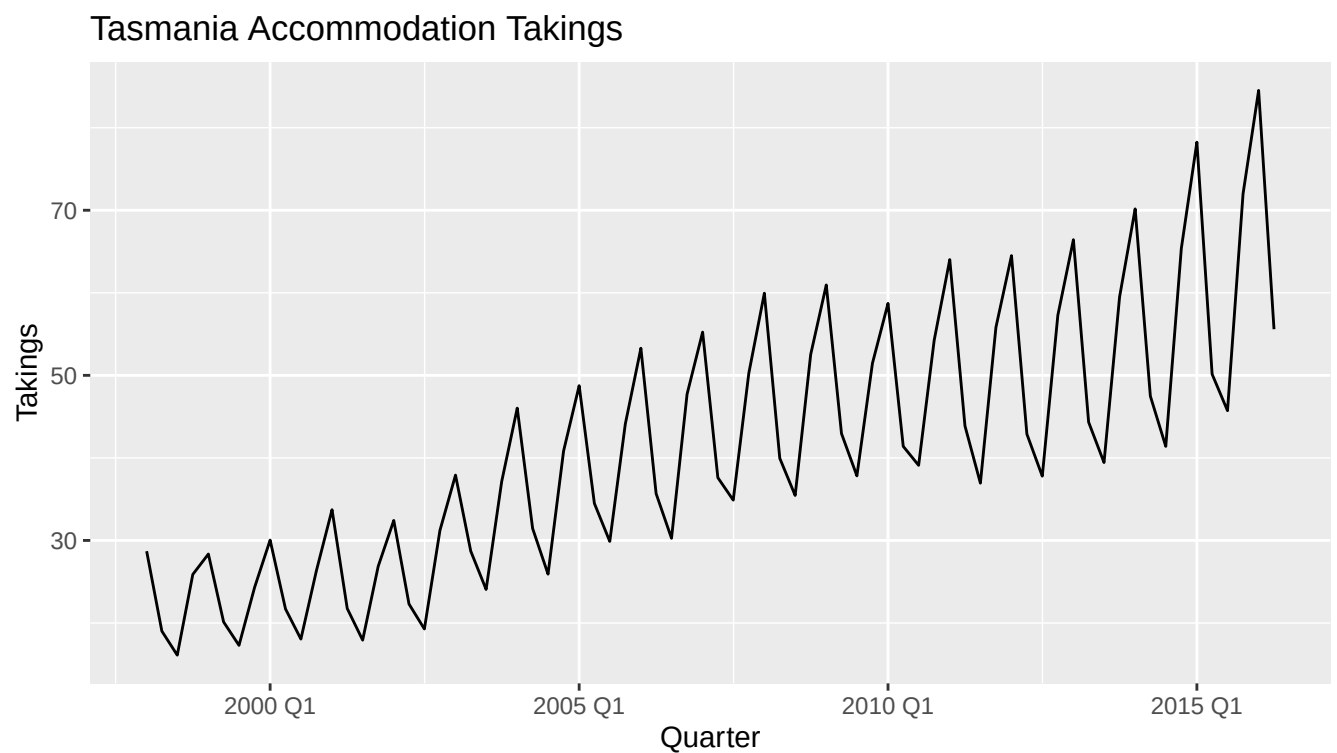
```
## # A tibble: 1 x 2
##   Country ndiffs
##   <fct>    <int>
## 1 Turkey      1
```

결론: $\lambda \approx 0.157$ 로 로그 변환에 가깝다.
1차 차분 후 정상성을 확보할 수 있다.

(b) Accommodation takings in Tasmania from `aus_accommodation`

```
tasmania <- aus_accommodation |>
  filter(State == "Tasmania")

tasmania |>
  autoplot(Takings) +
  labs(
    title = "Tasmania Accommodation Takings",
    y = "Takings", x = "Quarter"
  )
```



```
lambda_tas <- tasmania |>
  features(Takings, features = guerrero) |>
  pull(lambda_guerrero)
cat("Guerrero lambda:", round(lambda_tas, 3), "\n")
```

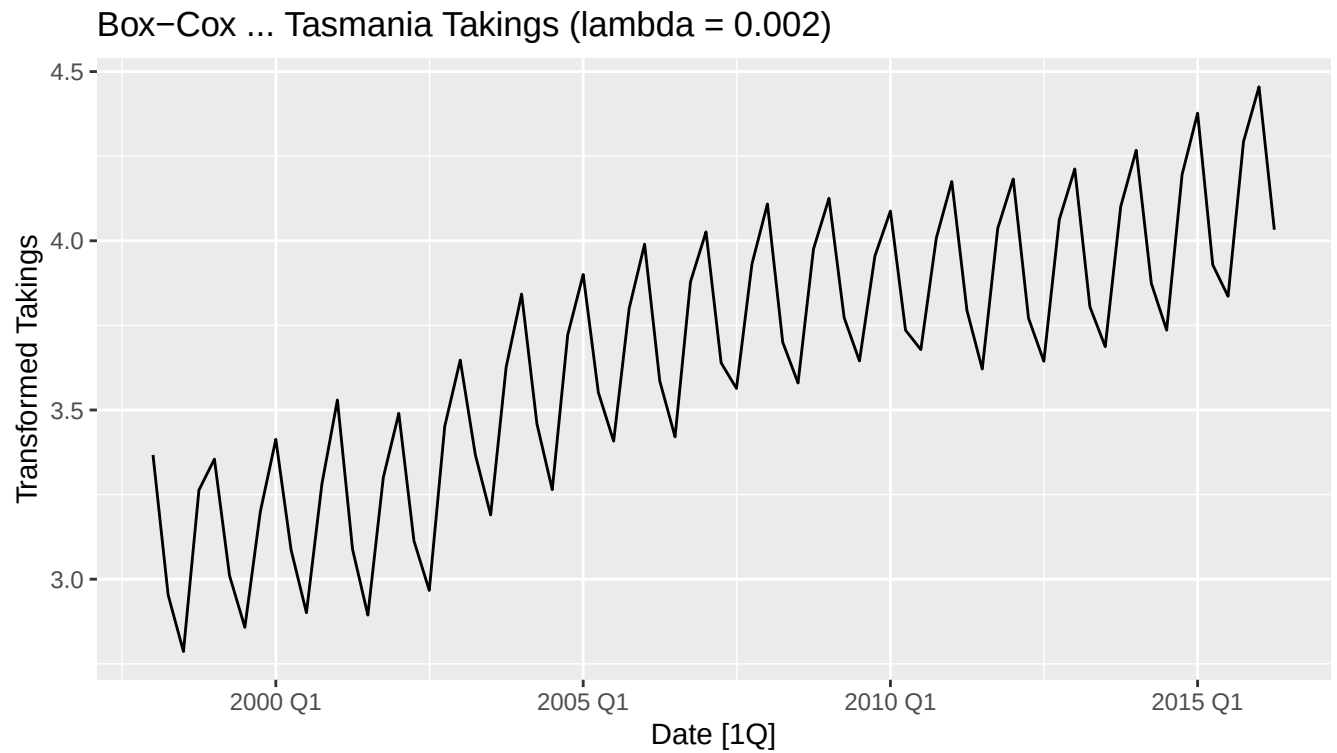
```
## Guerrero lambda: 0.002
```

```
tasmania |>
  mutate(Takings_bc = box_cox(Takings, lambda_tas)) |>
  autoplot(Takings_bc) +
  labs(
    title = paste0(
      "Box-Cox Tasmania Takings (lambda = ",
```

```

    round(lambda_tas, 3), ")")
  ),
  y = "Transformed Takings"
)

```



```

tasmania |>
  mutate(Takings_bc = box_cox(Takings, lambda_tas)) |>
  features(Takings_bc, unitroot_nsdiffs)

```

```

## # A tibble: 1 x 2
##   State    nsdiffs
##   <chr>    <int>
## 1 Tasmania      1

```

```

tasmania |>
  mutate(
    Takings_bc = box_cox(Takings, lambda_tas),
    Takings_sdiff = difference(Takings_bc, 4)
  ) |>
  features(Takings_sdiff, unitroot_ndiffs)

```

```

## # A tibble: 1 x 2
##   State    ndiffs
##   <chr>    <int>
## 1 Tasmania      0

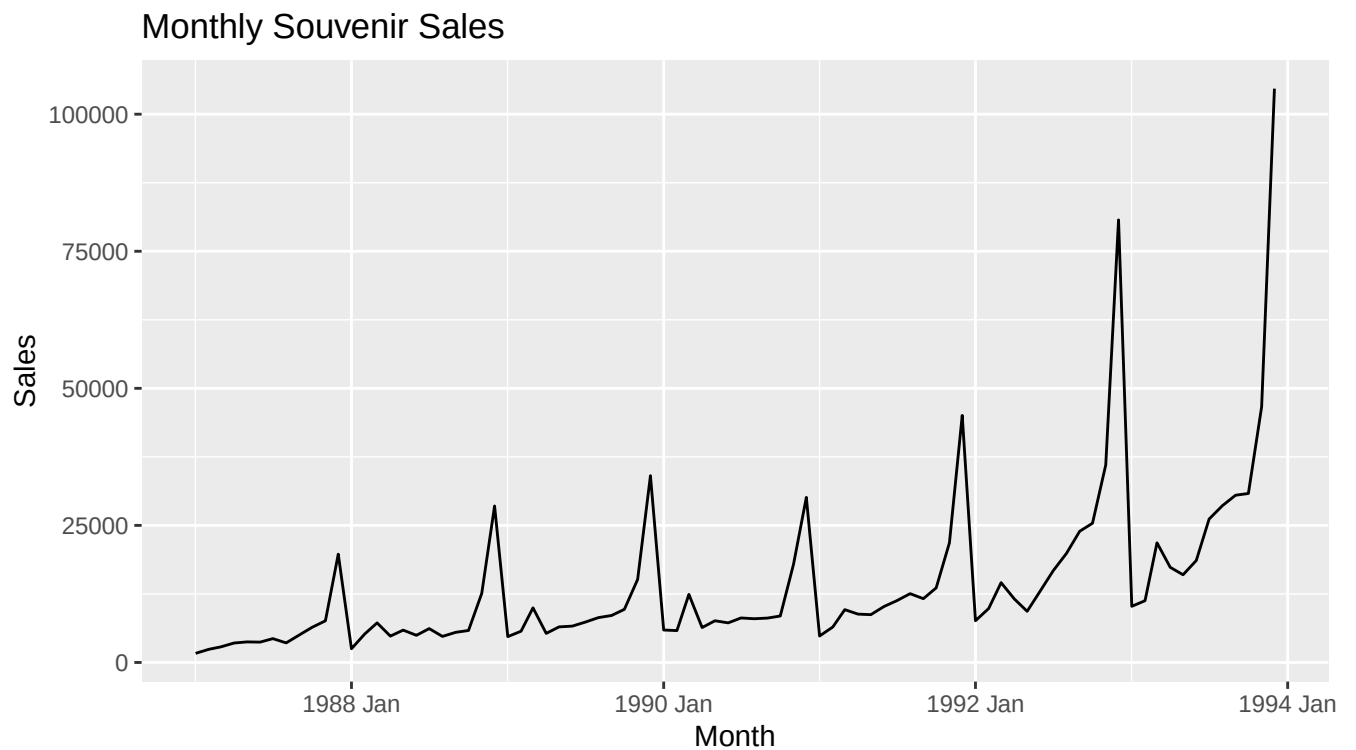
```

결론: $\lambda \approx 0.002$ 로 로그 변환에 해당한다.

분기 데이터이므로 계절 차분(lag=4) 1회 후 추가 차분 없이 정상성이 확보된다 (nsdiffs=1, ndiffs=0).

(c) Monthly sales from **souvenirs**

```
souvenirs |>
  autoplot(Sales) +
  labs(
    title = "Monthly Souvenir Sales",
    y = "Sales", x = "Month"
  )
```



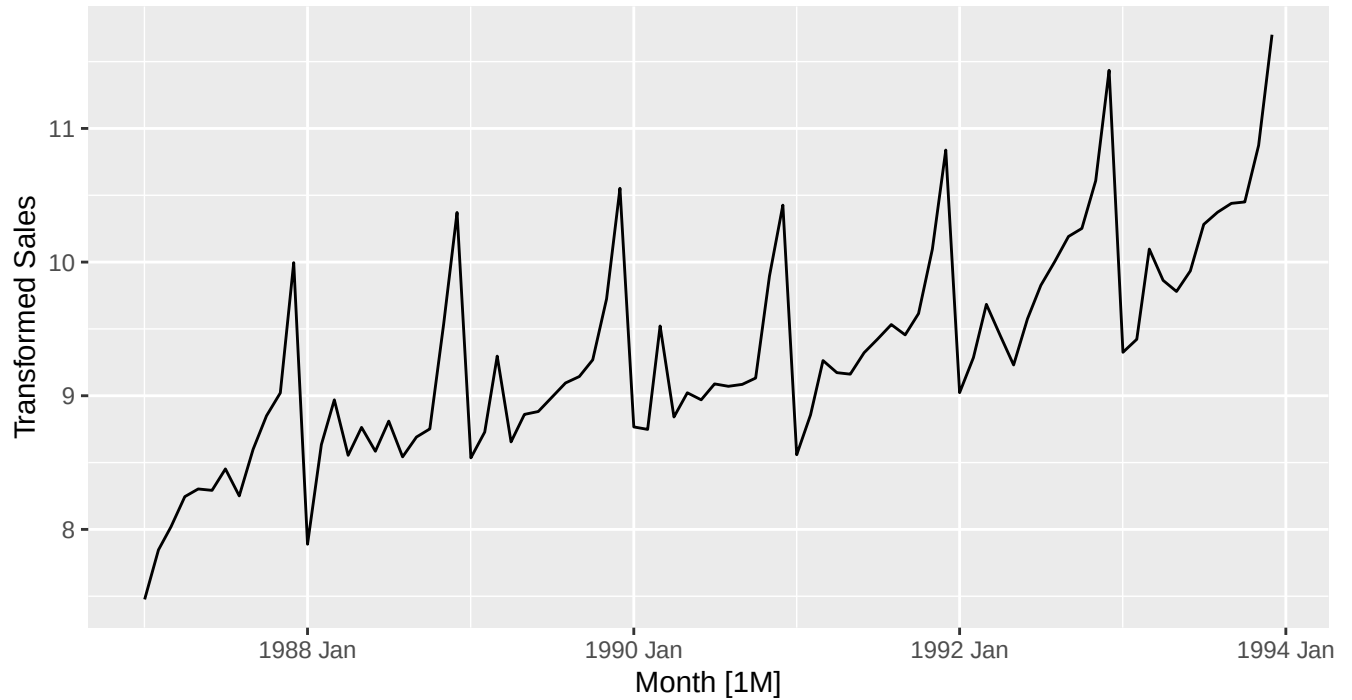
```
lambda_souv <- souvenirs |>
  features(Sales, features = guerrero) |>
  pull(lambda_guerrero)
cat("Guerrero lambda:", round(lambda_souv, 3), "\n")
```

```
## Guerrero lambda: 0.002
```

```
souvenirs |>
  mutate(Sales_bc = box_cox(Sales, lambda_souv)) |>
  autoplot(Sales_bc) +
```

```
labs(
  title = paste0(
    "Box-Cox Souvenir Sales (lambda = ",
    round(lambda_souv, 3), ")")
),
y = "Transformed Sales"
)
```

Box-Cox ... Souvenir Sales (lambda = 0.002)



```
souvenirs |>
  mutate(Sales_bc = box_cox(Sales, lambda_souv)) |>
  features(Sales_bc, unitroot_nsdiffs)
```

```
## # A tibble: 1 x 1
##   nsdiffs
##   <int>
## 1       1
```

```
souvenirs |>
  mutate(
    Sales_bc = box_cox(Sales, lambda_souv),
    Sales_sdiff = difference(Sales_bc, 12)
  ) |>
  features(Sales_sdiff, unitroot_ndiffs)
```

```
## # A tibble: 1 x 1
##   ndiffs
```

```
##      <int>
## 1      0
```

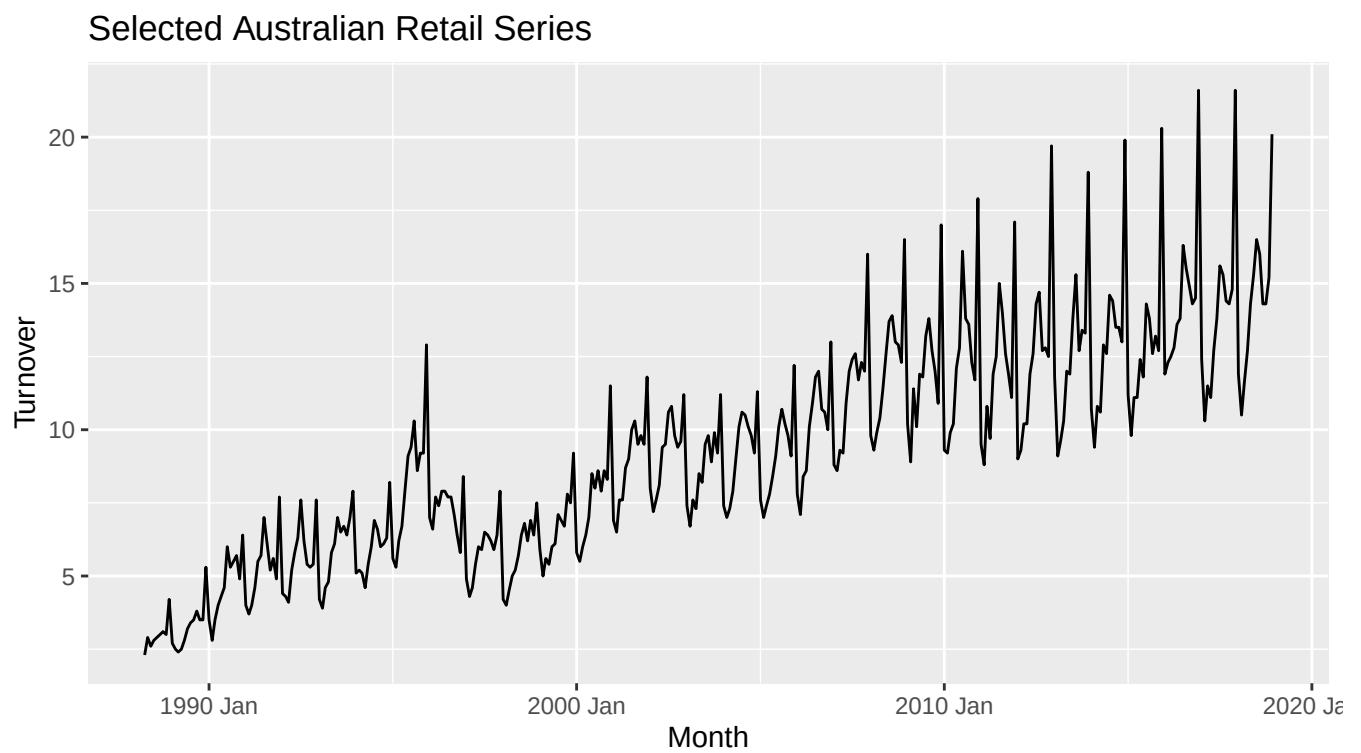
결론: $\lambda \approx 0.002$ 으로 로그 변환 적용. 계절 차분(lag=12) 1회 후 추가 차분 없이 정상성이 확보된다 (nsdiffs=1, ndiffs=0).

Exercise 5

Chapter 2 Exercise 7의 소매 데이터에서 적절한 차분 차수를 구하라.

```
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))

#
myseries |>
  autoplot(Turnover) +
  labs(
    title = "Selected Australian Retail Series",
    y = "Turnover", x = "Month"
  )
```

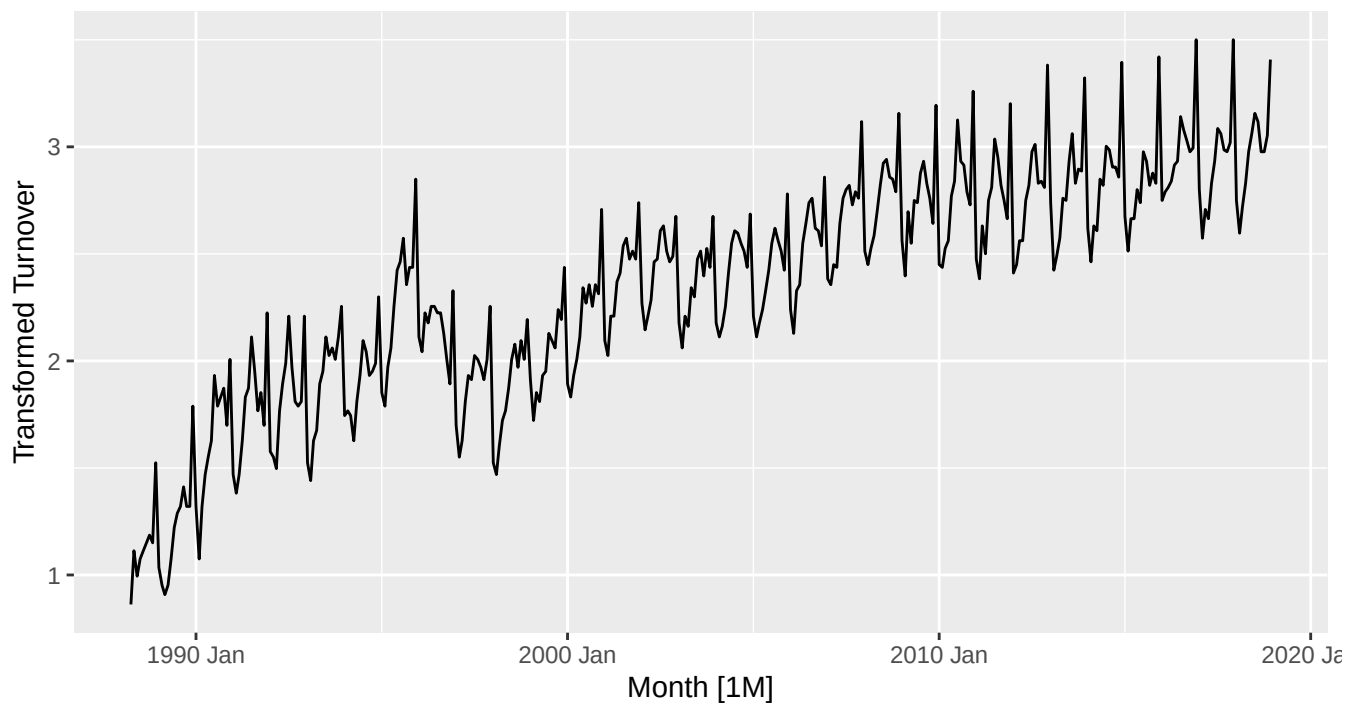



```
# Box-Cox
lambda_retail <- myseries |>
  features(Turnover, features = guerrero) |>
  pull(lambda_guerrero)
cat("Guerrero lambda:", round(lambda_retail, 3), "\n")
```

```
## Guerrero lambda: 0.083
```

```
myseries |>
  mutate(Turnover_bc = box_cox(Turnover, lambda_retail)) |>
  autoplot(Turnover_bc) +
  labs(
    title = paste0("Box-Cox (lambda = ", round(lambda_retail, 3), ")"),
    y = "Transformed Turnover"
  )
```

Box-Cox .. (lambda = 0.083)



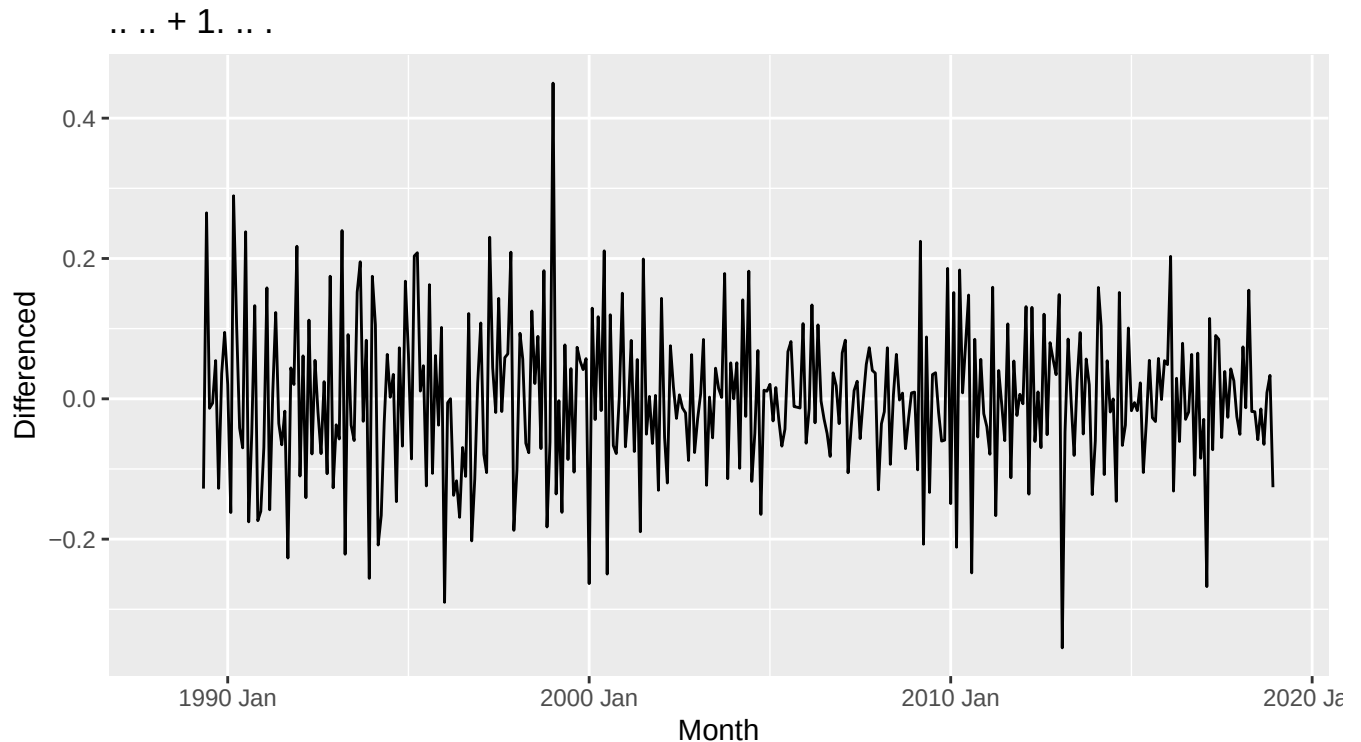
```
#
myseries |>
  mutate(T_bc = box_cox(Turnover, lambda_retail)) |>
  features(T_bc, unitroot_nsdiffs)
```

```
## # A tibble: 1 x 3
##   State          Industry          nsdiffs
##   <chr>          <chr>          <int>
## 1 Northern Territory Clothing, footwear and personal accessory retailing 1
```

```
#
myseries |>
  mutate(
    T_bc = box_cox(Turnover, lambda_retail),
    T_sdiff = difference(T_bc, 12)
  ) |>
  features(T_sdiff, unitroot_ndiffs)

## # A tibble: 1 x 3
##   State      Industry      ndiffs
##   <chr>      <chr>      <int>
## 1 Northern Territory Clothing, footwear and personal accessory retailing 0
```

```
myseries |>
  mutate(
    T_bc = box_cox(Turnover, lambda_retail),
    T_diff = difference(difference(T_bc, 12), 1)
  ) |>
  autoplot(T_diff) +
  labs(title = "      + 1      ", y = "Differenced", x = "Month")
```



결론: 실행 결과 nsdiffs = 1, ndiffs = 0이다.

즉 Box-Cox 변환($\lambda = 0.083$) 후 계절 차분($\text{lag}=12$) 1회만으로 정상성이 확보된다. 추가 1차 차분은 필요하지 않다.

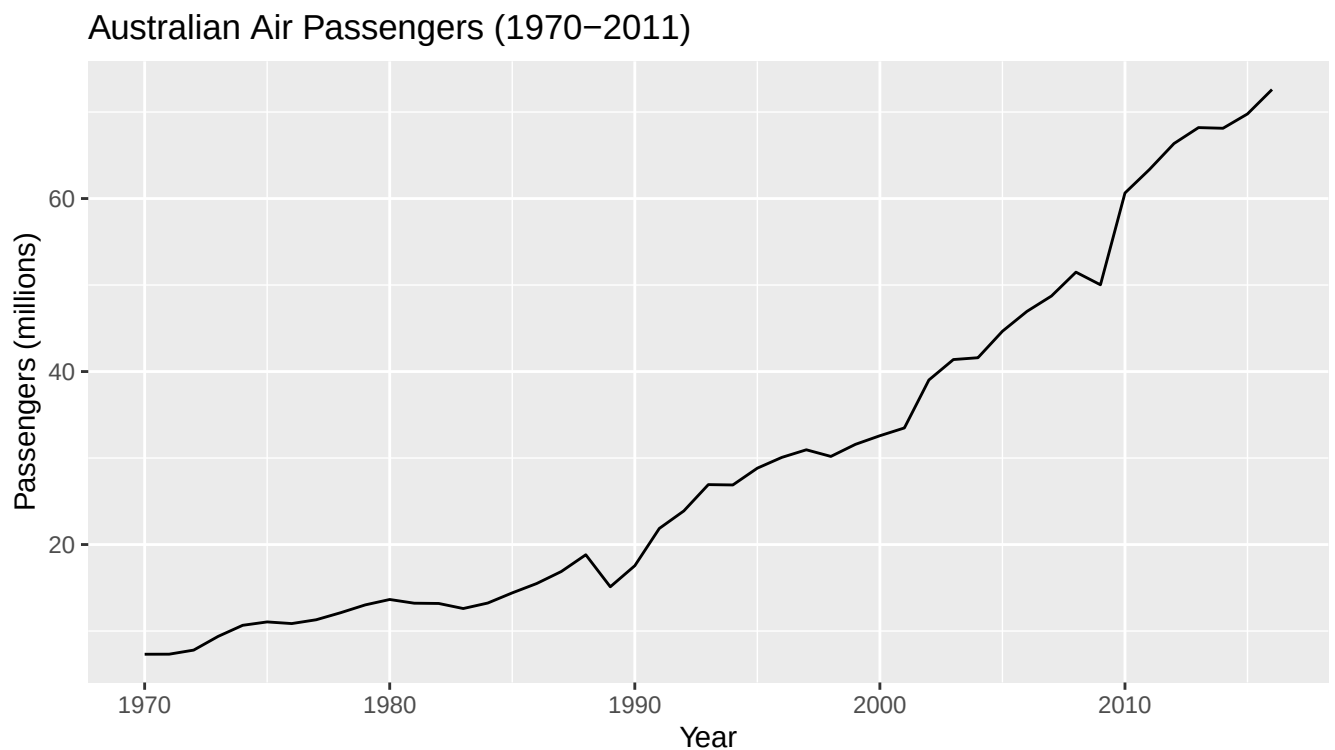
위 시각화는 시계열의 변화 양상을 파악하기 위해 계절 차분에 1차 차분을 추가로 적용한 결과이다 (과차분(over-differencing)에 해당).

Exercise 7

`aus_airpassengers` 데이터 (1970-2011년 호주 항공 승객 수)에 ARIMA 모델을 적합시켜라.

(a) 자동 ARIMA 모델 선택

```
aus_airpassengers |>
  autoplot(Passengers) +
  labs(
    title = "Australian Air Passengers (1970-2011)",
    y = "Passengers (millions)", x = "Year"
  )
```



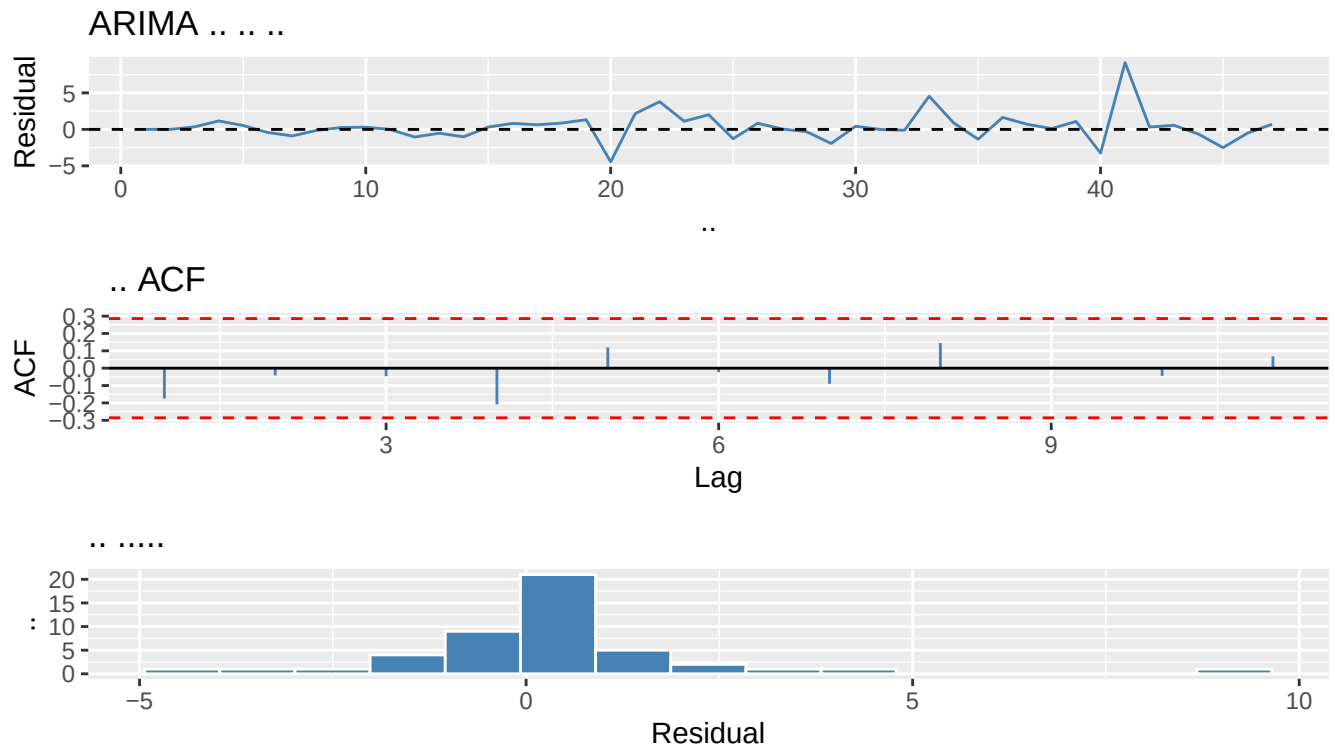
```
fit_air <- aus_airpassengers |>
  model(ARIMA = ARIMA(Passengers))

report(fit_air)
```

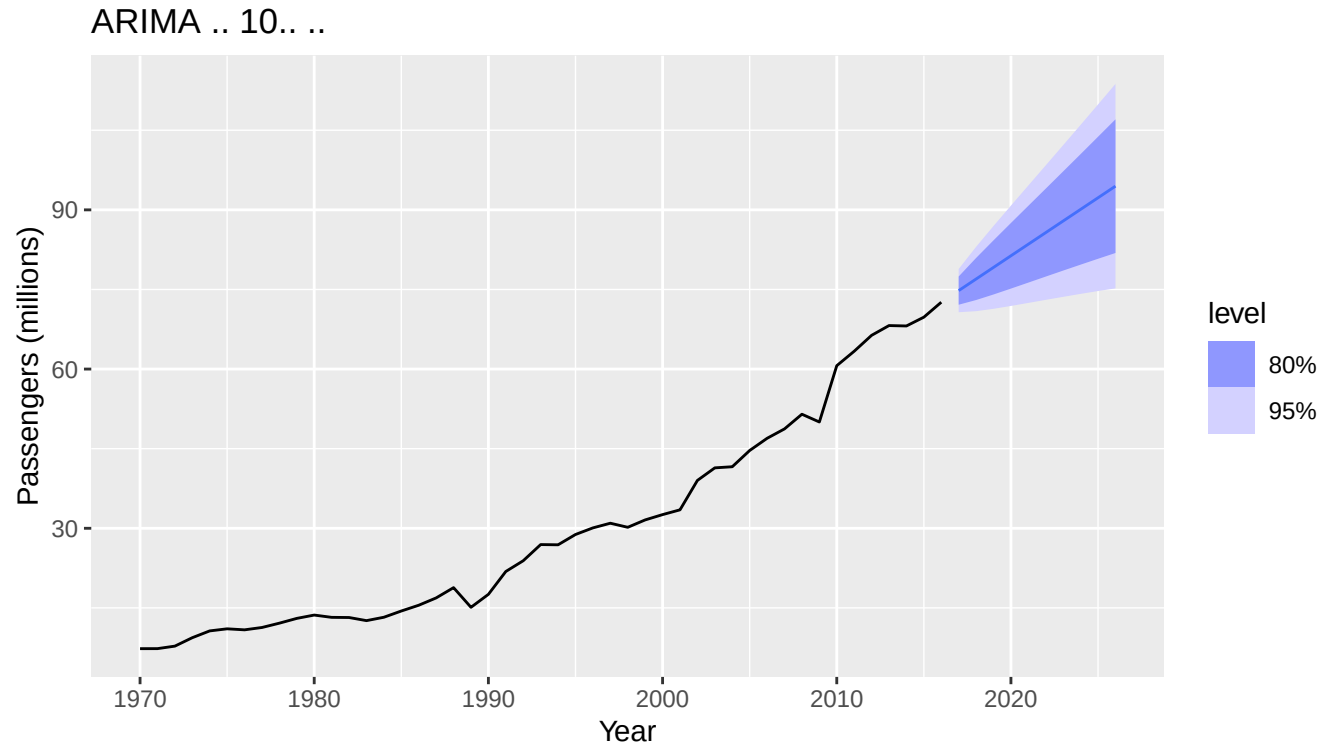
```
## Series: Passengers
## Model: ARIMA(0,2,1)
##
## Coefficients:
##          ma1
##       -0.8963
```

```
## s.e.    0.0594
##
## sigma^2 estimated as 4.308:  log likelihood=-97.02
## AIC=198.04   AICc=198.32   BIC=201.65
```

```
plot_residuals(fit_air, title = "ARIMA      ")
```



```
fit_air |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers) +
  labs(
    title = "ARIMA    10    ",
    y = "Passengers (millions)", x = "Year"
  )
```



(b) Backshift operator 표기

```
#           order
air_spec <- fit_air$ARIMA[[1]]$fit$spec
p_air <- air_spec$p
d_air <- air_spec$d
q_air <- air_spec$q
```

자동 선택된 모델은 ARIMA(0, 2, 1)이다.

```
params_air <- fit_air |> tidy()
params_air
```

```
## # A tibble: 1 x 6
##   .model term estimate std.error statistic p.value
##   <chr> <chr>   <dbl>    <dbl>    <dbl>   <dbl>
## 1 ARIMA  ma1     -0.896    0.0594   -15.1 3.24e-19
```

Backshift operator 표기:

선택 모델은 ARIMA(0, 2, 1)이므로 AR 항이 없고 MA(1) 항만 존재한다.

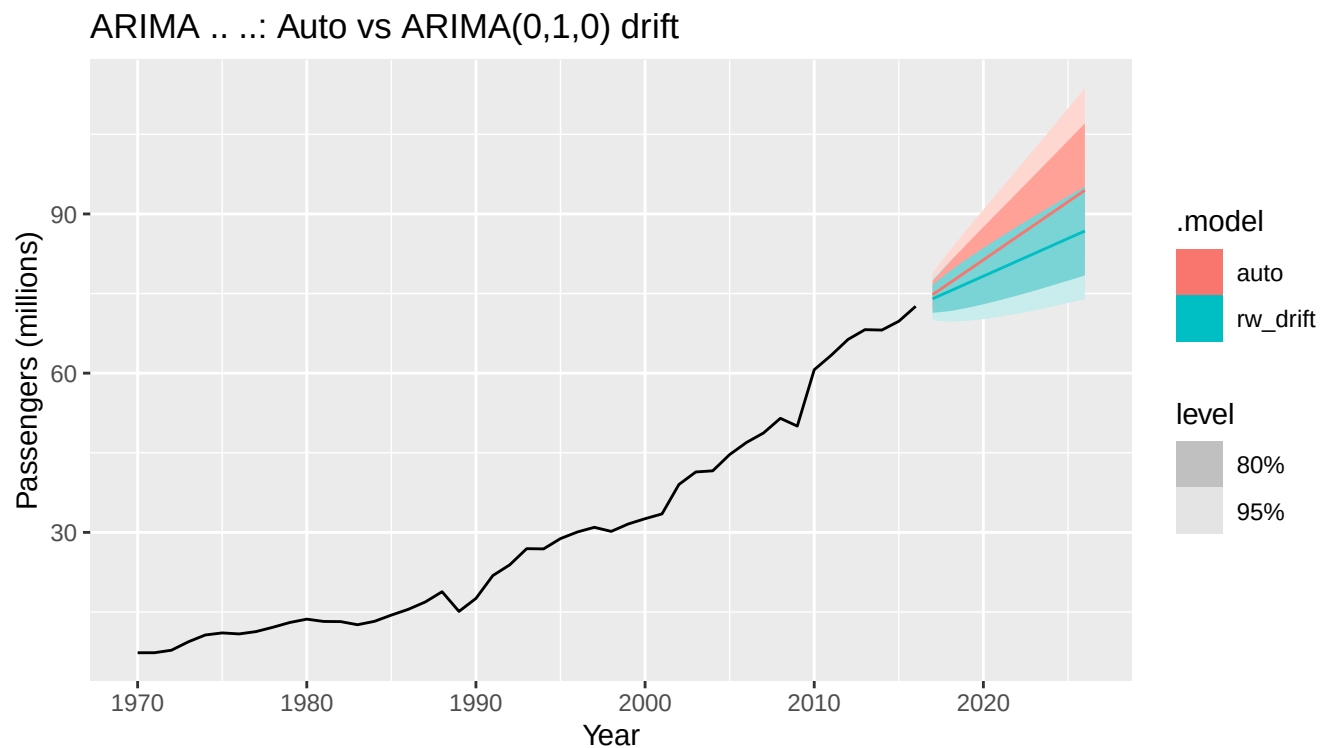
$$(1 - B)^2 y_t = (1 + (-0.8963) B) \varepsilon_t$$

파라미터 수치는 위 tidy() 출력값이 자동 대입된다. report(fit_air)로 전체 모델을 확인한다.

(c) ARIMA(0,1,0) with drift

```
fit_air_c <- aus_airpassengers |>
  model(
    auto = ARIMA(Passengers),
    rw_drift = ARIMA(Passengers ~ 1 + pdq(0, 1, 0))
  )

fit_air_c |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers) +
  labs(
    title = "ARIMA      : Auto vs ARIMA(0,1,0) drift",
    y = "Passengers (millions)", x = "Year"
  )
)
```



ARIMA(0,1,0) with drift는 단순 랜덤워크+드리프트 모델이다.
자동 선택 모델(ARIMA(0,2,1))과 비교하면, drift 모델은 선형 추세를 가정하는 반면 auto 모델은 2차 차분을 통해 가속 성장도 포착할 수 있어 예측 경로 및 구간의 폭이 다르게 나타난다.

(d) ARIMA(2,1,2) with drift

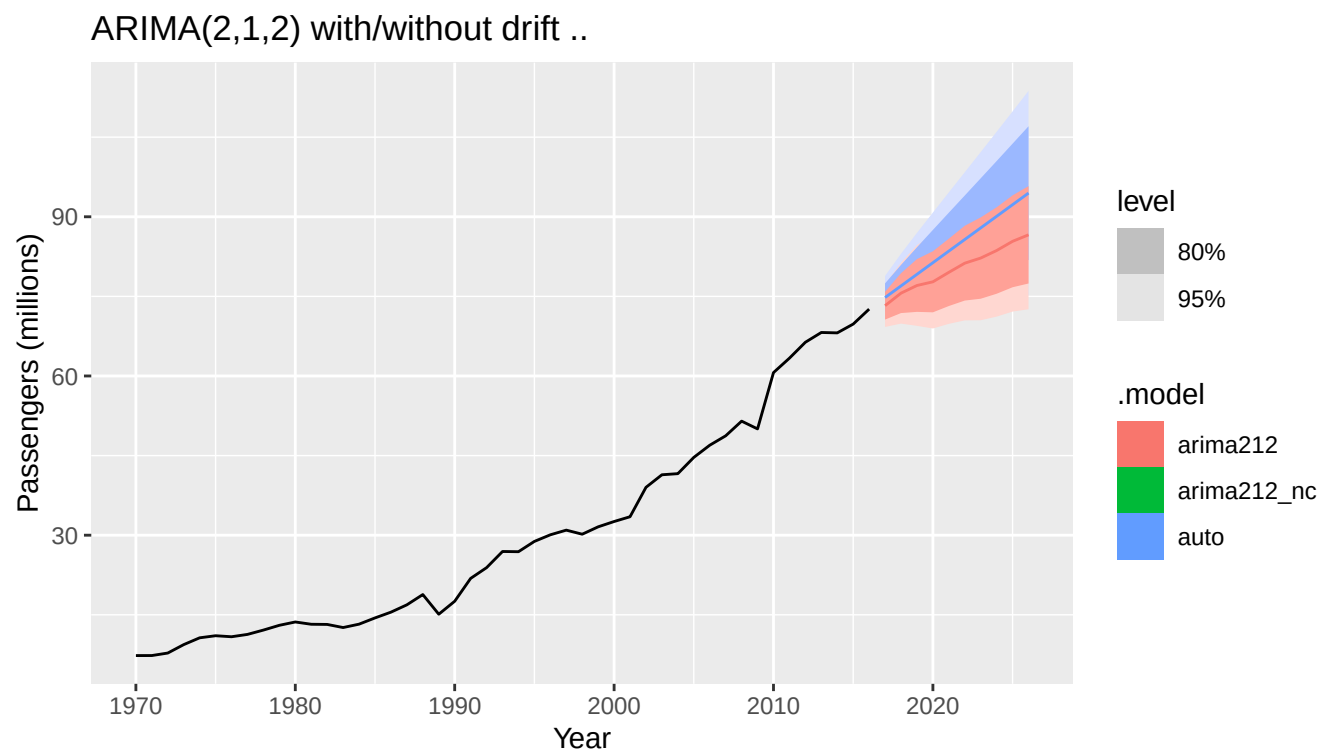
```
fit_air_d <- aus_airpassengers |>
  model(
```

```

auto = ARIMA(Passengers),
arima212 = ARIMA(Passengers ~ 1 + pdq(2, 1, 2)),
arima212_nc = ARIMA(Passengers ~ 0 + pdq(2, 1, 2))
)

fit_air_d |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers) +
  labs(
    title = "ARIMA(2,1,2) with/without drift ",
    y = "Passengers (millions)", x = "Year"
  )
)

```



상수항(~ 1)을 포함한 모델은 드리프트(장기 증가 추세)를 포착한다. 반면 상수항을 제거(~ 0)하면 장기 예측이 마지막 관측값 수준에서 정체되거나 불안정해질 수 있다.

(e) ARIMA(0,2,1) with constant

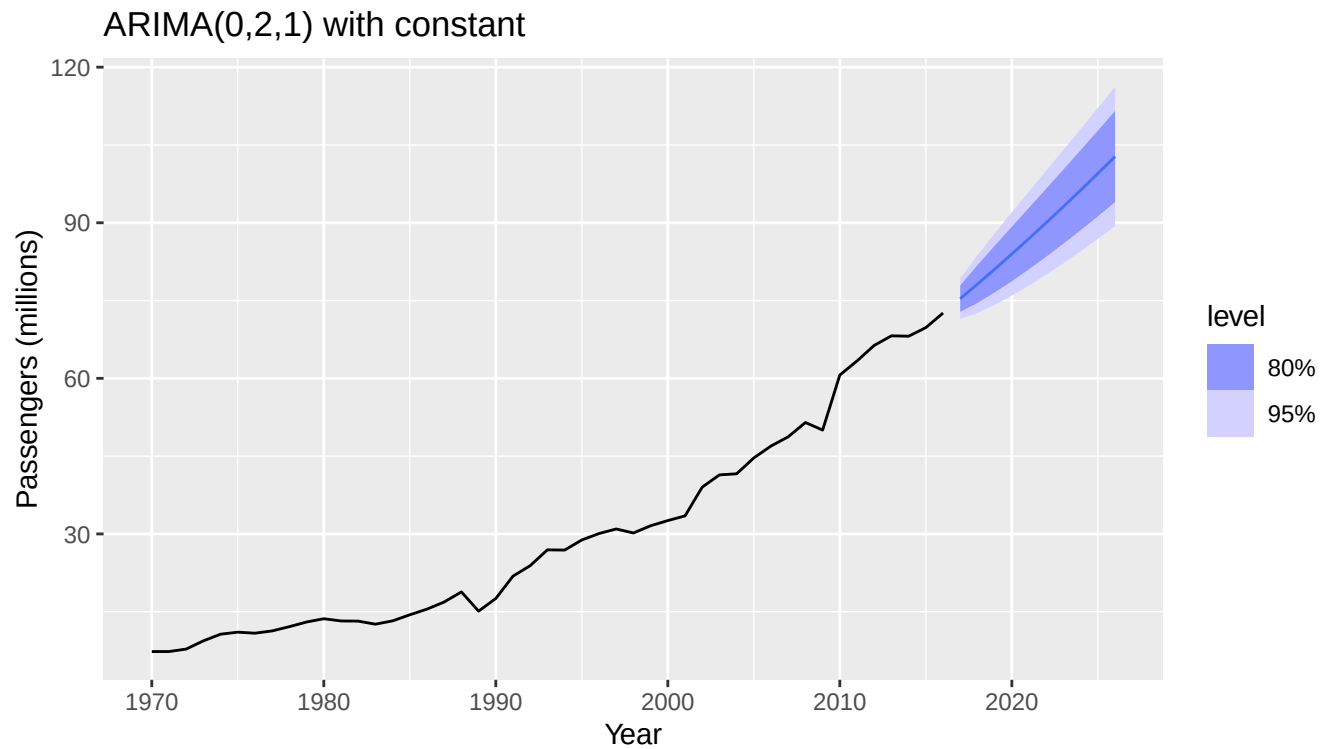
```

fit_air_e <- aus_airpassengers |>
  model(ARIMA(Passengers ~ 1 + pdq(0, 2, 1)))

fit_air_e |>
  forecast(h = 10) |>
  autoplot(aus_airpassengers) +

```

```
labs(
  title = "ARIMA(0,2,1) with constant",
  y = "Passengers (millions)", x = "Year"
)
```



ARIMA 모델에서 2차 차분($d = 2$) + 상수항 조합은 2차(quadratic) 추세를 의미하므로, 예측값이 급격하게 증가하는 경향이 생긴다. 이 데이터에서는 (a)의 자동 선택 모델인 ARIMA(0,2,1)에 상수항이 없는 것과 달리, 상수항을 추가함으로써 예측이 더욱 가파르게 증가함을 확인할 수 있다.

Exercise 9

aus_arrivals: 1981 Q1 - 2012 Q3 호주 방문 국제 방문객 수 (분기별)

(a) 한 국가 선택 및 시계열 시각화

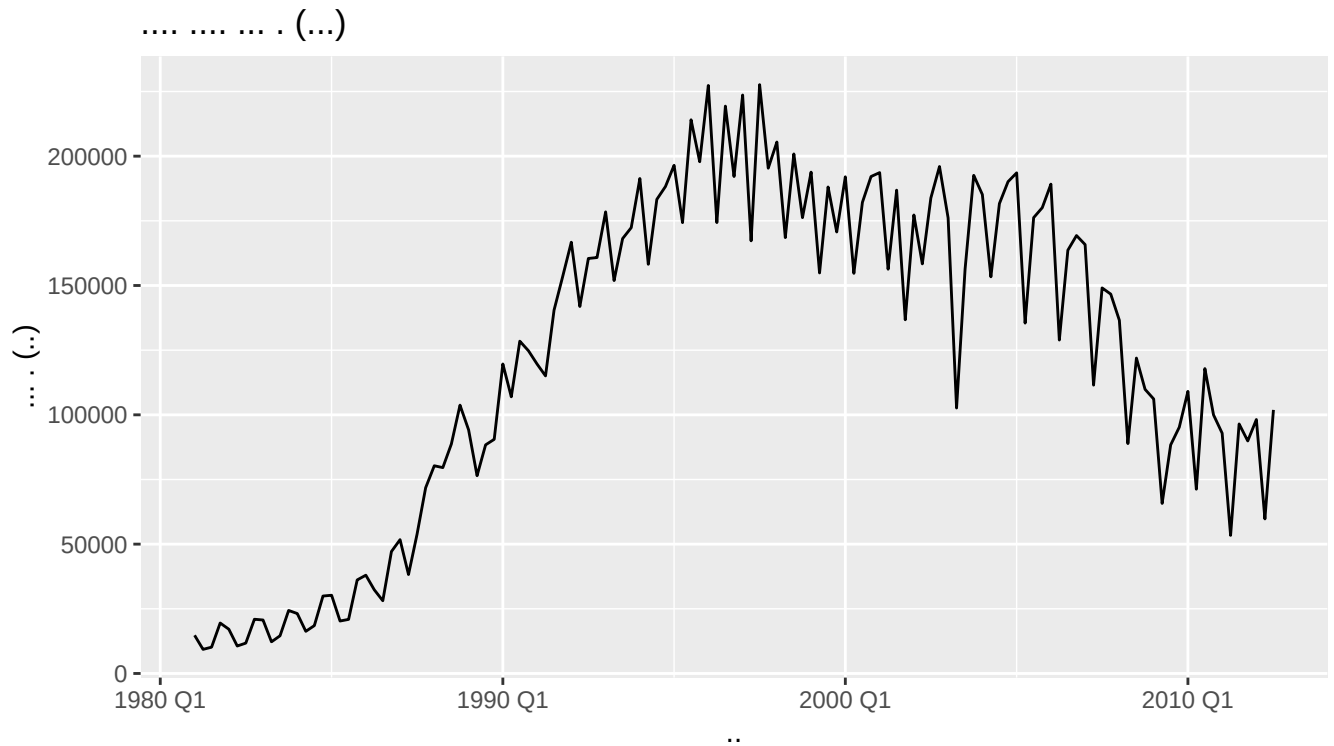
Japan을 선택한다.

```
japan_arrivals <- aus_arrivals |>
  filter(Origin == "Japan")

japan_arrivals |>
```



```
autoplot(Arrivals) +
  labs(
    title = "      ( )",
    y = "      ( )", x = " "
  )
```



시계열 특성:
1980년대 중반부터 2000년 초반까지 증가 추세가 있었으나,
이후 감소 추세로 전환되었다. 뚜렷한 계절성도 관찰된다.

(b) 차분으로 정상성 확보

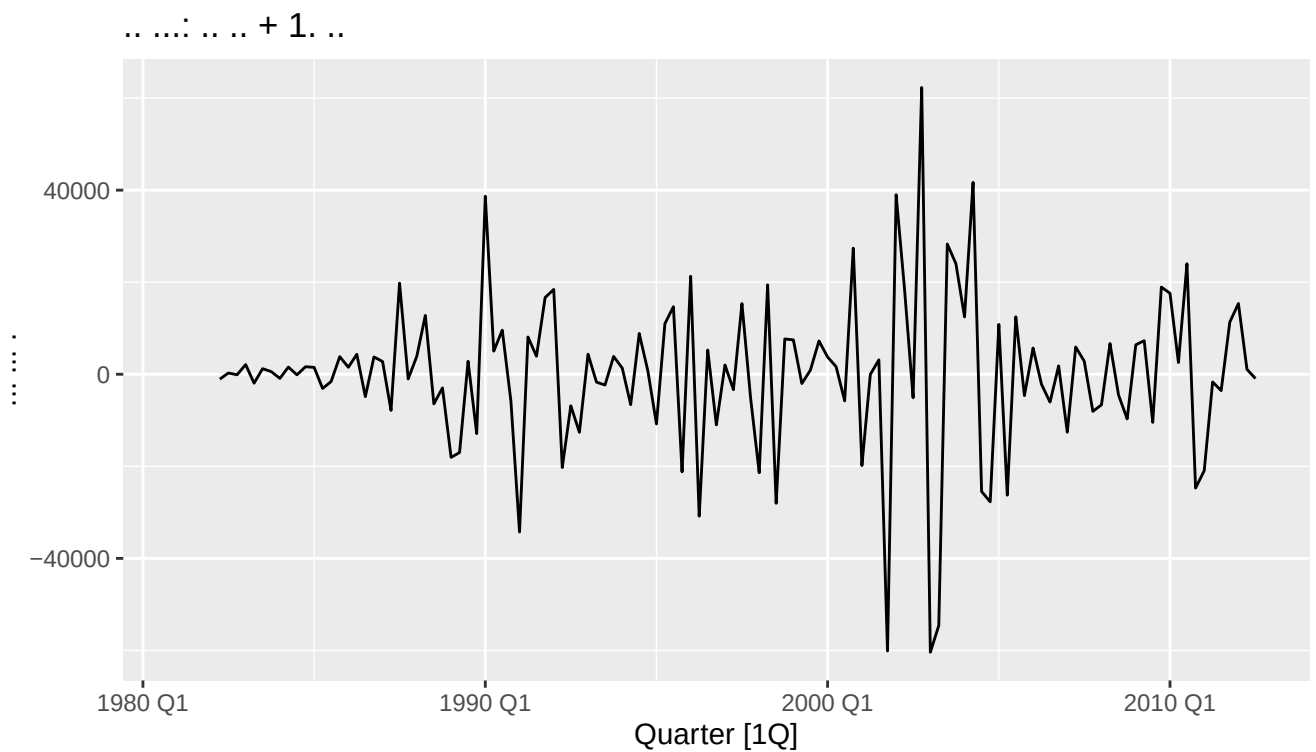
```
#
japan_arrivals |>
  features(Arrivals, unitroot_nsdiffs)
```

```
## # A tibble: 1 x 2
##   Origin nsdiffs
##   <chr>    <int>
## 1 Japan      1
```

```
japan_arrivals |>
  mutate(Arrivals_sdif = difference(Arrivals, 4)) |>
  features(Arrivals_sdif, unitroot_nsdiffs)
```

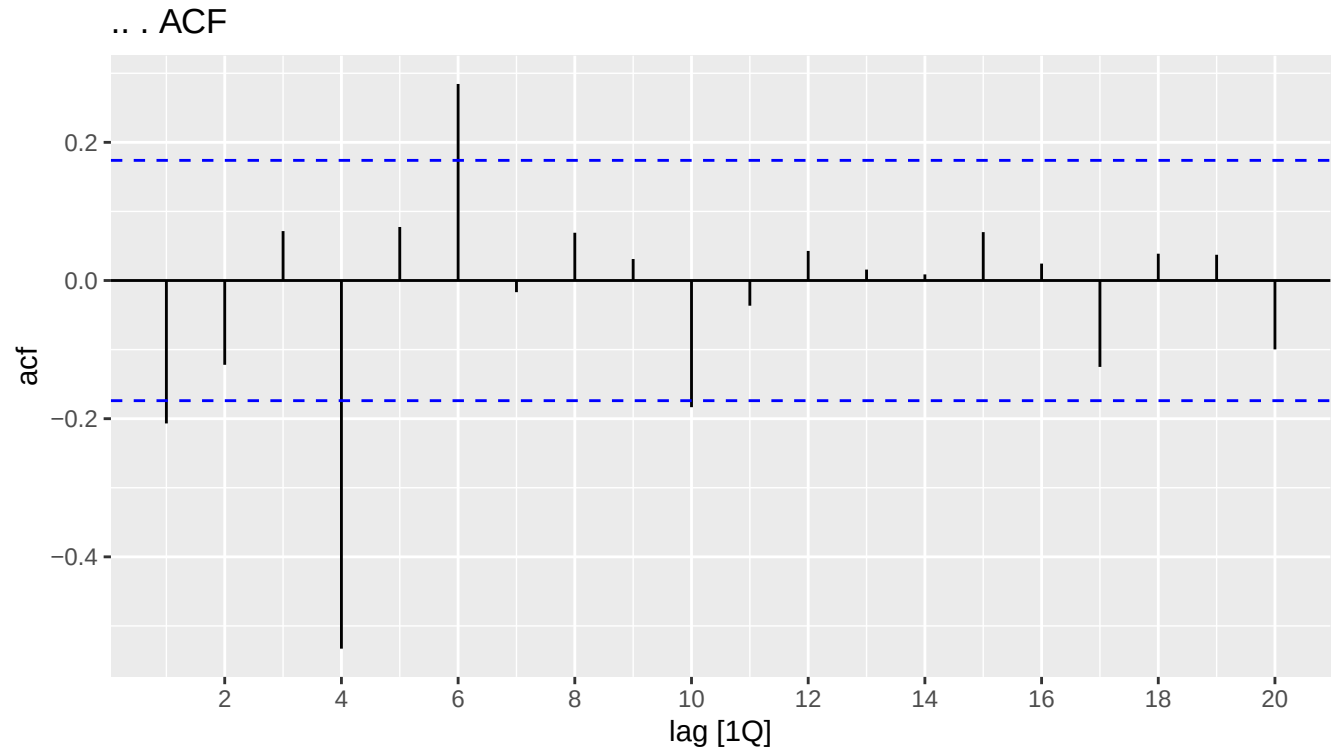
```
## # A tibble: 1 x 2
##   Origin ndiffs
##   <chr>   <int>
## 1 Japan     1
```

```
japan_arrivals |>
  mutate(Arrivals_diff = difference(difference(Arrivals, 4), 1)) |>
  autoplot(Arrivals_diff) +
  labs(
    title = "      :      + 1  ",
    y = "      "
  )
```



(c) 차분 후 ACF 분석

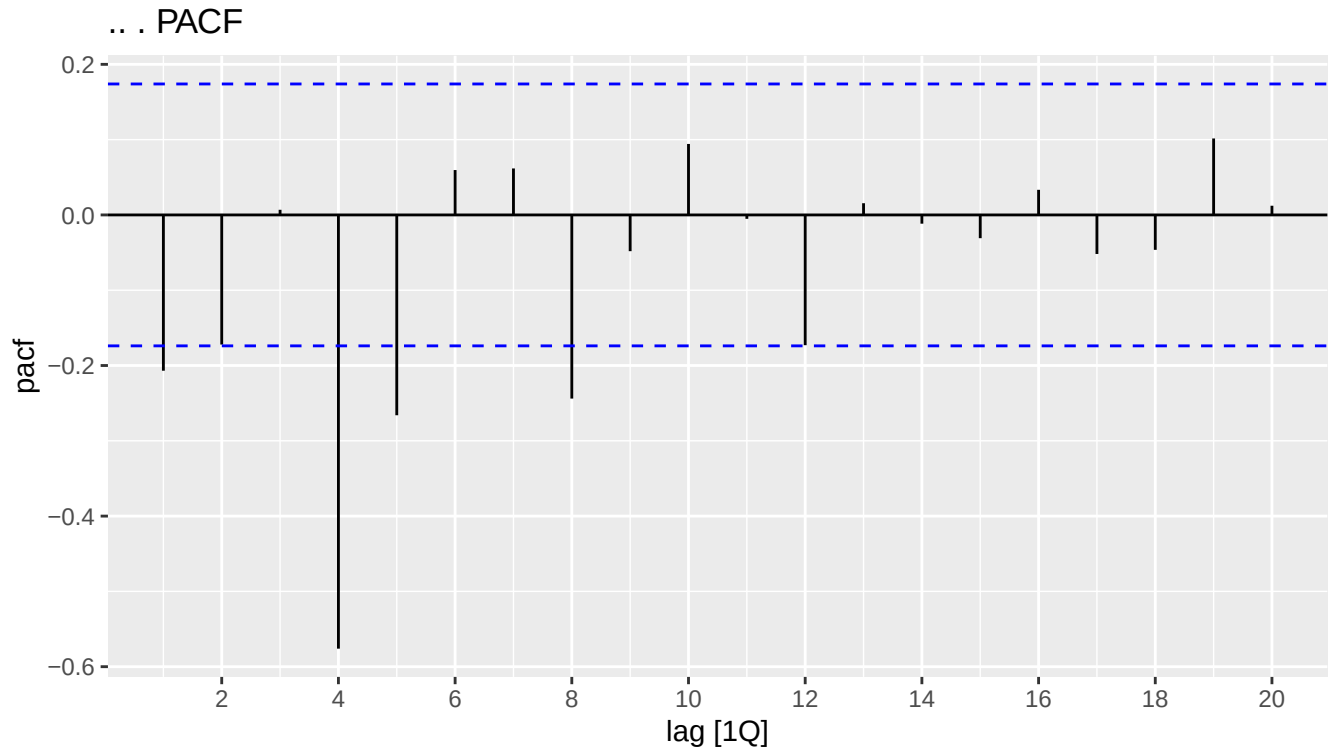
```
japan_arrivals |>
  mutate(Arrivals_diff = difference(difference(Arrivals, 4), 1)) |>
  ACF(Arrivals_diff) |>
  autoplot() +
  labs(title = "      ACF")
```



lag-1에서 유의한 음의 스파이크 이후 급격히 절단(cut-off)되어 MA(1) 성분을 시사하며, lag-4(계절 lag)에서도 유의한 음의 스파이크가 관찰되어 계절 MA(1) 성분을 시사한다.

(d) 차분 후 PACF 분석

```
japan_arrivals |>
  mutate(Arrivals_diff = difference(difference(Arrivals, 4), 1)) |>
  PACF(Arrivals_diff) |>
  autoplot() +
  labs(title = "    PACF")
```



lag-1에서 유의한 음의 스파이크가 있고 이후 절단되어 AR(1) 성분 가능성을 시사하며, lag-2에서도 유의한 값이 나타나 AR(2) 가능성도 있다. lag-4(계절 lag)에서도 유의한 값이 관찰되어 계절 AR(1) 가능성을 시사한다.

(e) 추천 모델

위 ACF/PACF 플롯을 바탕으로 모델을 추론한다.

비계절 성분 (d=1, D=1 차분 후 기준):

- ACF: lag-1에서 유의한 음의 스파이크 이후 급격히 절단 → MA(1) 성분 시사
- PACF: lag-1~2에서 유의한 값이 나타나고 이후 감소 → AR(1) 또는 AR(2) 가능성

계절 성분 (lag = 4, 8 기준):

- ACF: lag-4(계절 lag)에서 유의한 음의 스파이크 → 계절 MA(1) 성분 시사
- PACF: 계절 lag에서 점진적 감소 → 계절 MA 쪽이 더 우세

따라서 추천 모델은 $ARIMA(1, 1, 1)(0, 1, 1)_4$ 또는 $ARIMA(2, 1, 0)(0, 1, 1)_4$ 이다. 아래 AICc 비교를 통해 최종 모델을 결정한다.

```
fit_japan_cand <- japan_arrivals |>
  model(
    cand1 = ARIMA(Arrivals ~ pdq(1, 1, 1) + PDQ(0, 1, 1)),
    cand2 = ARIMA(Arrivals ~ pdq(2, 1, 0) + PDQ(0, 1, 1)),
    cand3 = ARIMA(Arrivals ~ pdq(0, 1, 1) + PDQ(0, 1, 1))
  )
```

```
)

glance(fit_japan_cand) |> select(.model, AIC, AICc, BIC)
```

```
## # A tibble: 3 x 4
##   .model   AIC  AICc   BIC
##   <chr> <dbl> <dbl> <dbl>
## 1 cand1 2670. 2670. 2681.
## 2 cand2 2674. 2675. 2686.
## 3 cand3 2670. 2670. 2678.
```

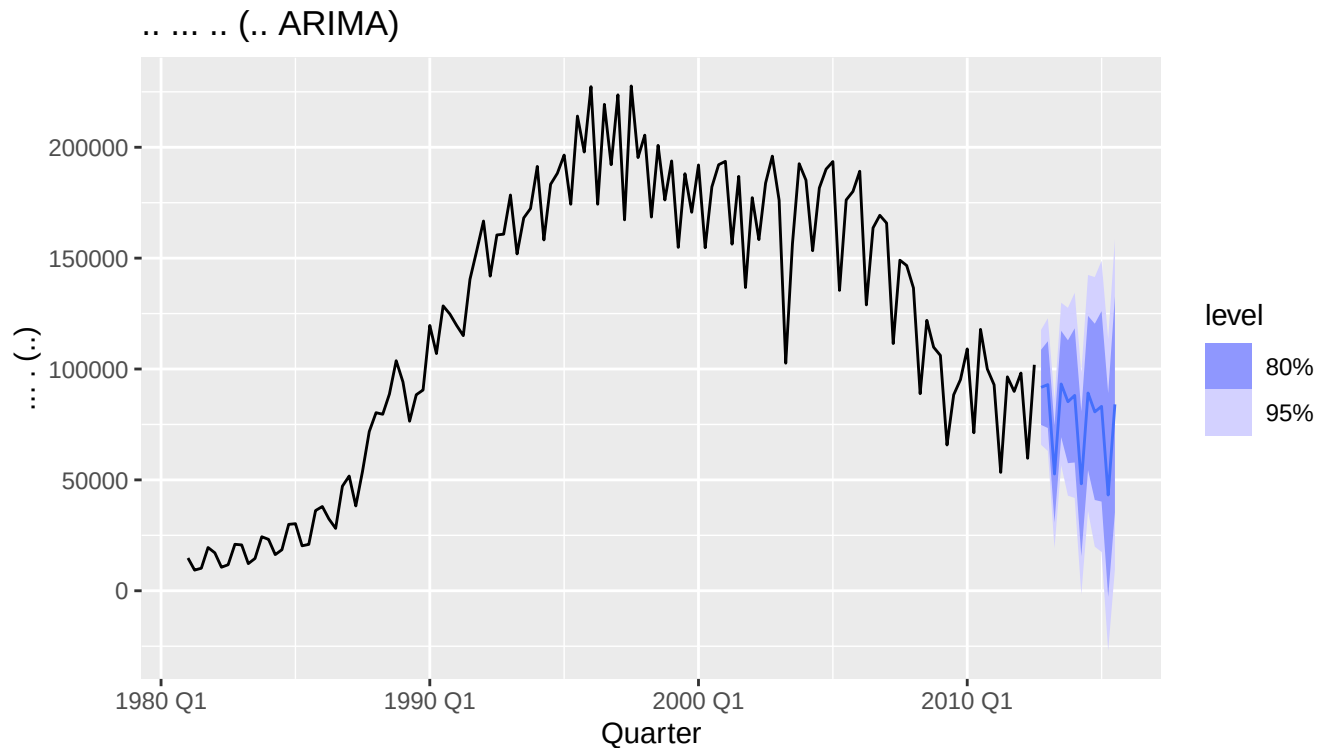
(f) 자동 ARIMA와 비교

```
fit_japan <- japan_arrivals |>
  model(auto = ARIMA(Arrivals))

report(fit_japan)
```

```
## Series: Arrivals
## Model: ARIMA(0,1,1)(1,1,1)[4]
##
## Coefficients:
##          ma1      sar1      sma1
##      -0.4228  -0.2305  -0.5267
## s.e.   0.0944   0.1337   0.1246
##
## sigma^2 estimated as 174801727:  log likelihood=-1330.66
## AIC=2669.32  AICc=2669.66  BIC=2680.54
```

```
fit_japan |>
  forecast(h = 12) |>
  autoplot(japan_arrivals) +
  labs(
    title = "      ( ARIMA)",
    y = "      ( )"
  )
```



자동 선택 결과: 자동 ARIMA가 선택한 모델은 $ARIMA(0,1,1)(1,1,1)[4]$ 이다.

모델 비교 판단:

- 9(e)에서 수동으로 검토한 후보 중 AICc 기준 최적은 cand1 ($AICc = 2670.1$)이다.
- 자동 ARIMA의 $AICc = 2669.66$ 이다.
- 자동 선택 모델이 9(e) 최적 후보와 같다면 → 수동 분석과 자동 탐색이 일치하므로 해당 모델에 대한 신뢰도가 높다.
- 자동 선택 모델이 9(e) 최적 후보와 다르다면 → 자동 ARIMA가 더 넓은 범위를 탐색하여 더 낮은 $AICc$ 를 달성한 것이므로, $AICc$ 가 더 낮은 자동 선택 모델이 더 낫다고 판단한다.
- 일반적으로 $ARIMA()$ 자동 탐색은 $AICc$ 를 최소화하도록 최적화되어 있으므로, 동일 기준에서는 자동 선택 모델이 수동 후보 이상의 성능을 가진다.

(g) Backshift operator 표기

```
japan_spec <- fit_japan$auto[[1]]$fit$spec
p_j <- japan_spec$p
d_j <- japan_spec$d
q_j <- japan_spec$q
P_j <- japan_spec$P
D_j <- japan_spec$D
Q_j <- japan_spec$Q
sp_j <- japan_spec$period

params_japan <- fit_japan |> tidy()
params_japan
```

```
## # A tibble: 3 x 7
##   Origin .model term estimate std.error statistic p.value
##   <chr>   <chr> <chr>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 Japan auto   ma1     -0.423    0.0944   -4.48 0.0000171
## 2 Japan auto   sar1    -0.231    0.134    -1.72 0.0872
## 3 Japan auto   sma1    -0.527    0.125    -4.23 0.0000458
```

자동 선택된 모델은 ARIMA(0,1,1)(1,1,1)[4]이다.

Backshift operator 표기:

$$(1 - B)^1(1 - B^4)^1(1 - (-0.2305)B^4)y_t = (1 + (-0.4228)B)(1 + (-0.5267)B^4)\varepsilon_t$$

수치 대입 전개식 ($w_t = (1 - B)(1 - B^4)y_t$ 로 정의할 때):

$$w_t - (-0.2305)w_{t-4} = \varepsilon_t + (-0.4228)\varepsilon_{t-1} + (-0.5267)\varepsilon_{t-4} + (0.2227)\varepsilon_{t-5}$$

여기서 w_t 는 계절 차분 및 1차 차분 후 시계열이며, 실제 수치는 tidy() 출력을 참조한다.

Exercise 11

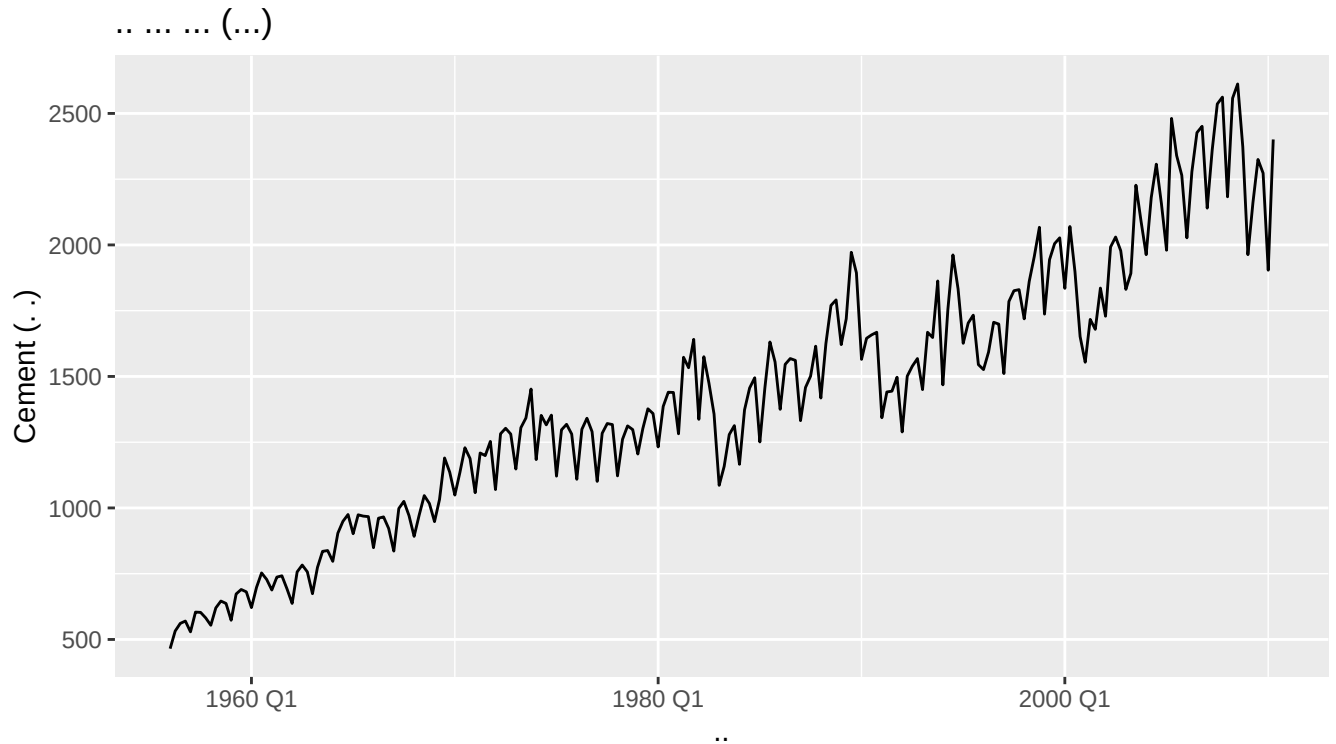
aus_production에서 전기(electricity), 시멘트(cement), 가스(gas) 중 하나를 선택하여 계절 ARIMA를 적합시켜라.

Cement를 선택한다.

(a) 변환 필요 여부

```
cement <- aus_production |>
  select(Quarter, Cement) |>
  filter(!is.na(Cement))

cement |>
  autoplot(Cement) +
  labs(
    title = "      ( )",
    y = "Cement ( )", x = " "
  )
```



```
lambda_cement <- cement |>
  features(Cement, features = guerrero) |>
  pull(lambda_guerrero)
cat("Guerrero lambda:", round(lambda_cement, 3), "\n")
```

```
## Guerrero lambda: -0.309
```

실행 결과 $\lambda \approx -0.309$ 이므로 Box-Cox 변환을 적용한다.
($\lambda < 0$ 이므로 역수 변환에 가까우며, 분산 안정화 효과를 기대할 수 있다.)

(b) 정상성 확인 및 차분

```
cement |>
  mutate(Cement_bc = box_cox(Cement, lambda_cement)) |>
  features(Cement_bc, unitroot_nsdiffs)
```

```
## # A tibble: 1 x 1
##   nsdiffs
##   <int>
## 1      1
```

```
cement |>
  mutate(
    Cement_bc = box_cox(Cement, lambda_cement),
    Cement_sdiff = difference(Cement_bc, 4)
```



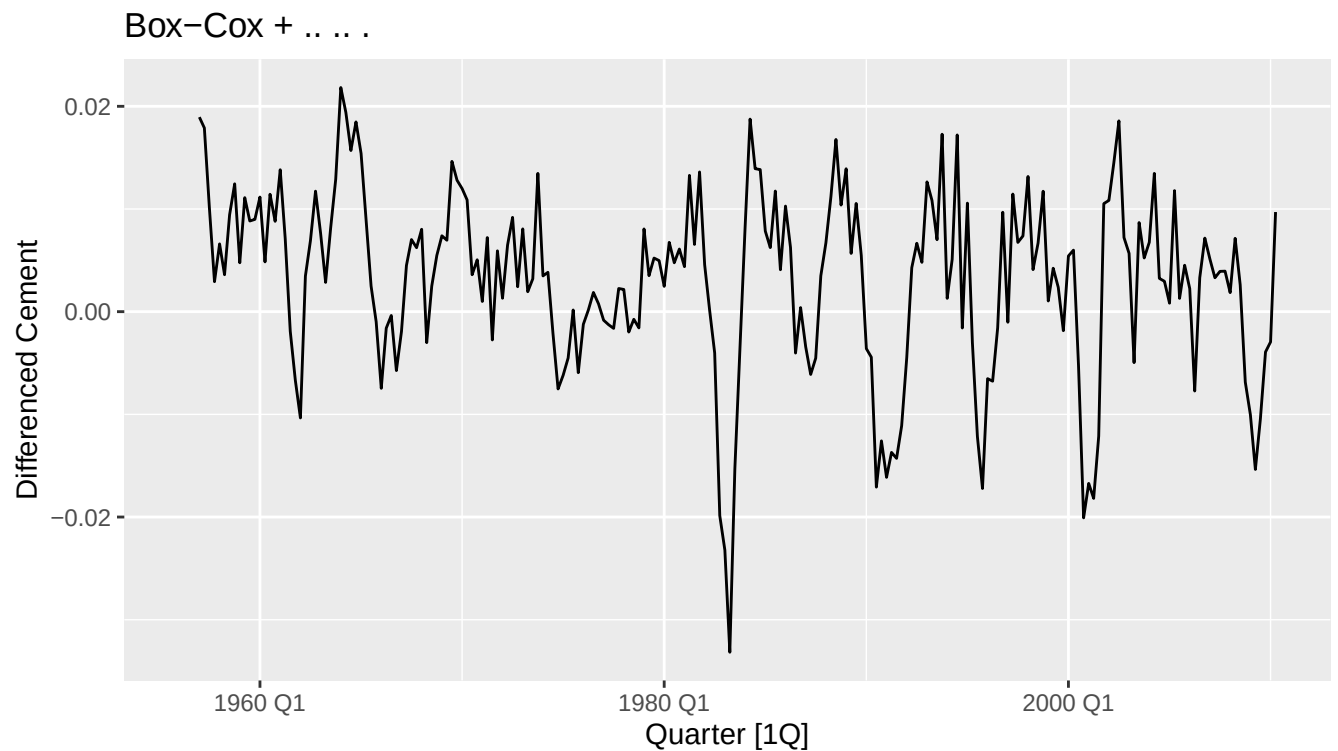
```

) |>
features(Cement_sdif, unitroot_ndiffs)

## # A tibble: 1 x 1
##   ndiffs
##   <int>
## 1     0

cement |>
  mutate(
    Cement_bc = box_cox(Cement, lambda_cement),
    Cement_diff = difference(Cement_bc, 4)
  ) |>
  autoplot(Cement_diff) +
  labs(title = "Box-Cox + ", y = "Differenced Cement")

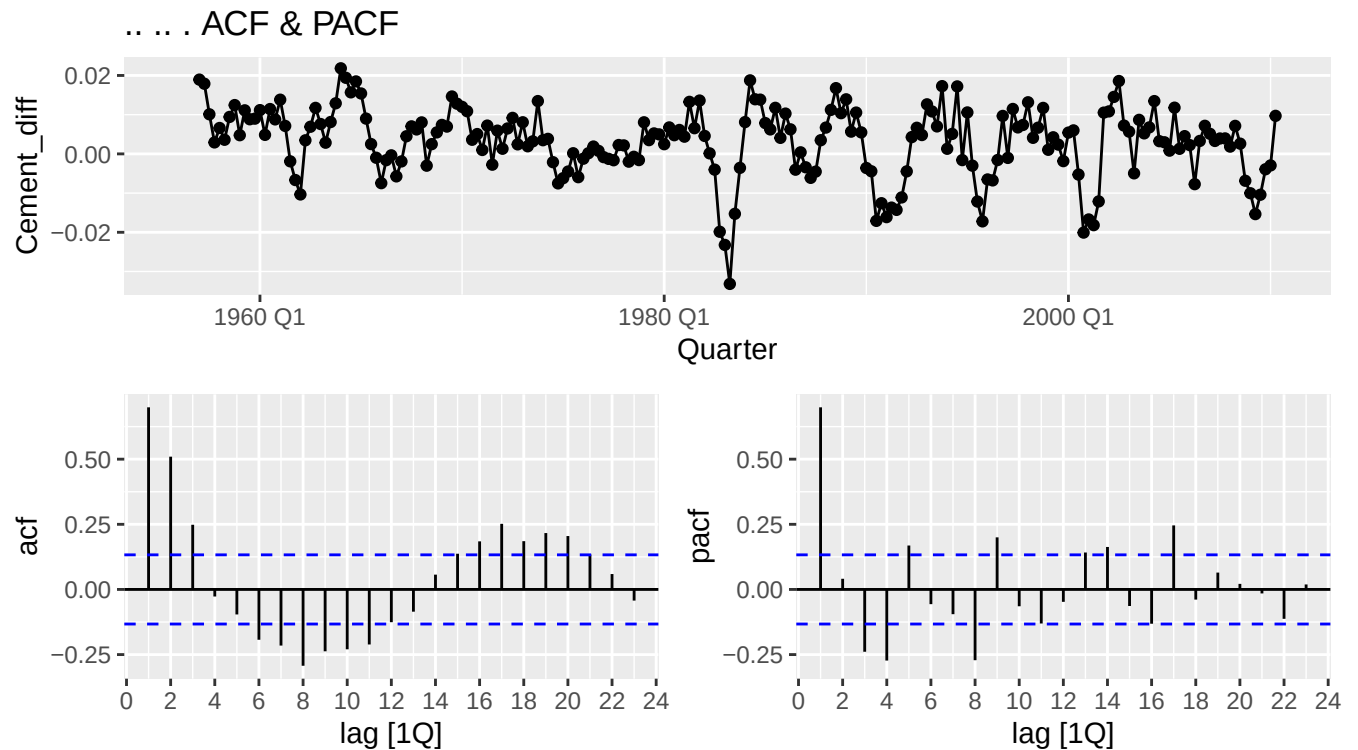
```



결론: nsdiffs = 1이므로 계절 차분(lag=4) 1회가 필요하고, 이후 ndiffs = 0이므로 추가 차분은 불필요하다.
즉 Box-Cox 변환 후 계절 차분 1회만으로 정상성이 확보된다.

(c) 모델 후보 식별

```
# ACF/PACF
cement |>
  mutate(
    Cement_bc = box_cox(Cement, lambda_cement),
    Cement_diff = difference(Cement_bc, 4)
  ) |>
  gg_tsdisplay(Cement_diff, plot_type = "partial") +
  labs(title = "      ACF & PACF")
```



```
fit_cement <- cement |>
  model(
    auto = ARIMA(box_cox(Cement, lambda_cement)),
    m1 = ARIMA(box_cox(Cement, lambda_cement) ~ pdq(1, 0, 1) + PDQ(1, 1, 1)),
    m2 = ARIMA(box_cox(Cement, lambda_cement) ~ pdq(0, 0, 1) + PDQ(0, 1, 1))
  )

glance(fit_cement) |> select(.model, AIC, AICc, BIC)
```

```
## # A tibble: 3 x 4
##   .model    AIC    AICc    BIC
##   <chr>    <dbl> <dbl> <dbl>
## 1 auto    -1643. -1642. -1623.
## 2 m1      -1638. -1638. -1618.
## 3 m2      -1512. -1512. -1498.
```

AIC 기반 모델 비교:

문제에서 요구하는 기준은 AIC이다. AIC는 $-2\log L + 2k$ 로 정의되며, 값이 작을수록 더 좋은 모델을 의미한다.

AICc는 소표본 보정 AIC로, 관측치 수가 충분히 클 때는 AIC와 거의 동일하게 움직인다.

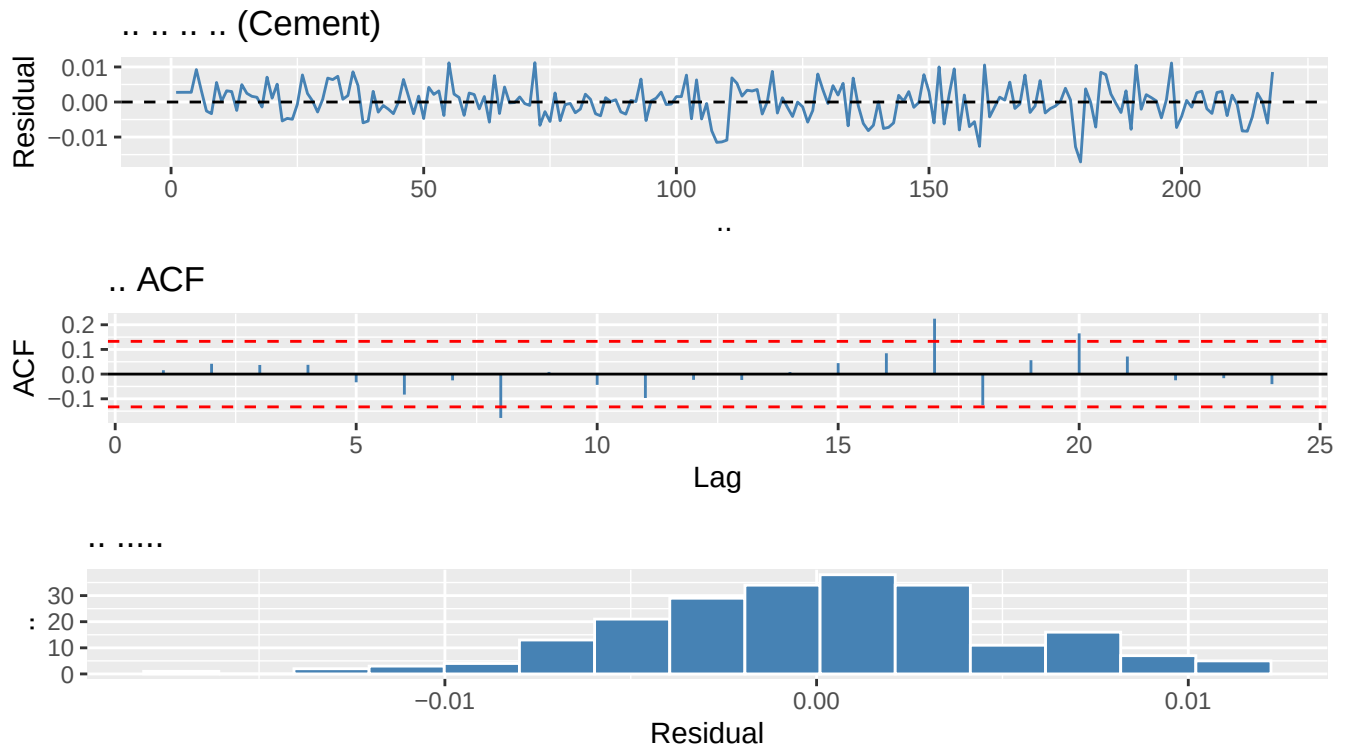
- 세 모델 중 AIC가 가장 낮은 모델을 최적 모델로 선택한다.
- `auto` 모델은 `ARIMA()` 함수가 AICc를 최소화하며 자동 탐색한 결과이므로 일반적으로 AIC도 가장 낮거나 이에 준하는 값을 가진다.
- `m1(pdq(1,0,1)+PDQ(1,1,1))`과 `m2(pdq(0,0,1)+PDQ(0,1,1))`는 수작업 후보 모델로, AIC 값을 `auto`와 비교하여 자동 선택 결과의 타당성을 검증한다.

(d) 잔차 진단

```
best_cement <- fit_cement |> select(auto)
report(best_cement)
```

```
## Series: Cement
## Model: ARIMA(2,0,1)(0,1,1)[4] w/ drift
## Transformation: box_cox(Cement, lambda_cement)
##
## Coefficients:
##          ar1      ar2      ma1      sma1  constant
##          0.3271  0.6148  0.4295 -0.8887      2e-04
## s.e.      0.1595  0.1510  0.1793   0.0480      1e-04
##
## sigma^2 estimated as 2.574e-05:  log likelihood=827.45
## AIC=-1642.9   AICc=-1642.49   BIC=-1622.7
```

```
plot_residuals(best_cement, title = "          (Cement)")
```

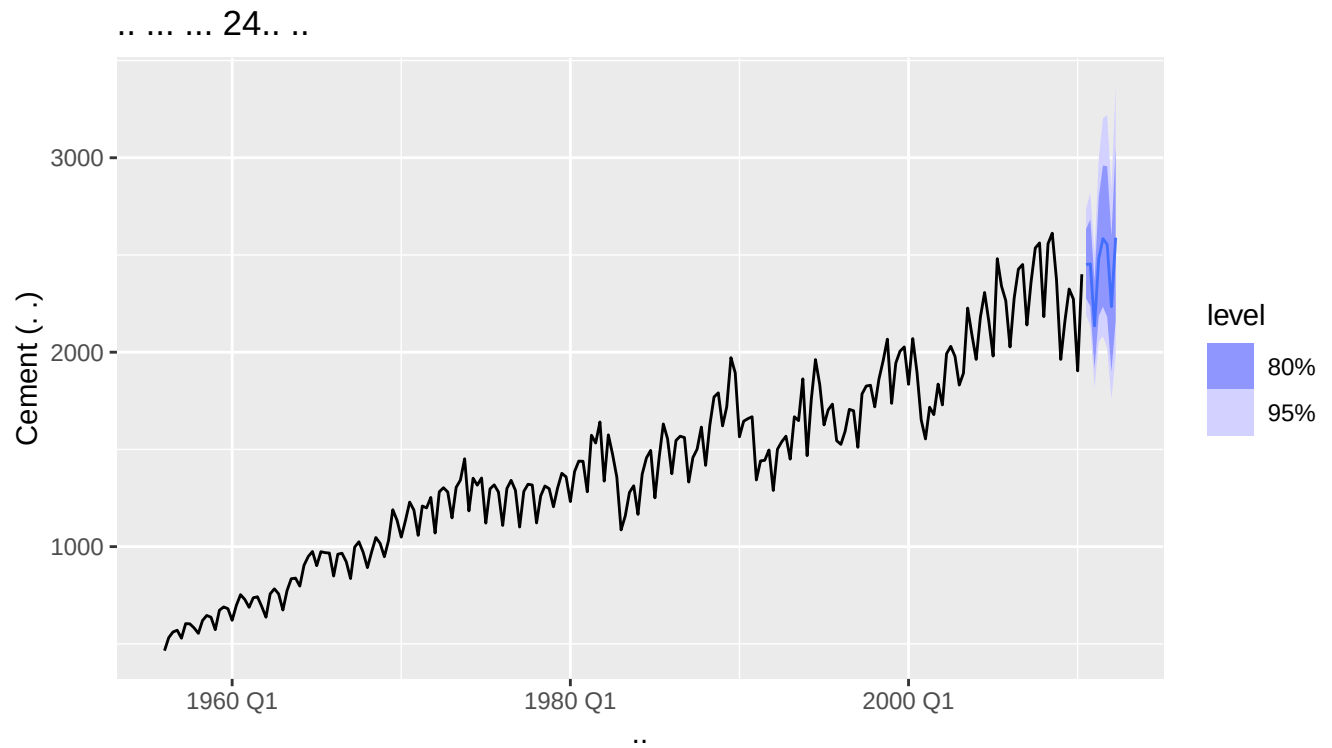


잔차 진단 판단:

잔차 시계열은 뚜렷한 패턴 없이 0 주변에서 무작위로 변동하고, ACF에서 임계값을 벗어나는 유의한 스파이크가 관찰되지 않으며, 히스토그램은 정규분포에 가까운 형태를 보인다.
따라서 잔차는 백색잡음으로 판단하며, 선택된 모델이 데이터를 적절히 설명하고 있다고 결론 내린다.

(e) 24개월 예측

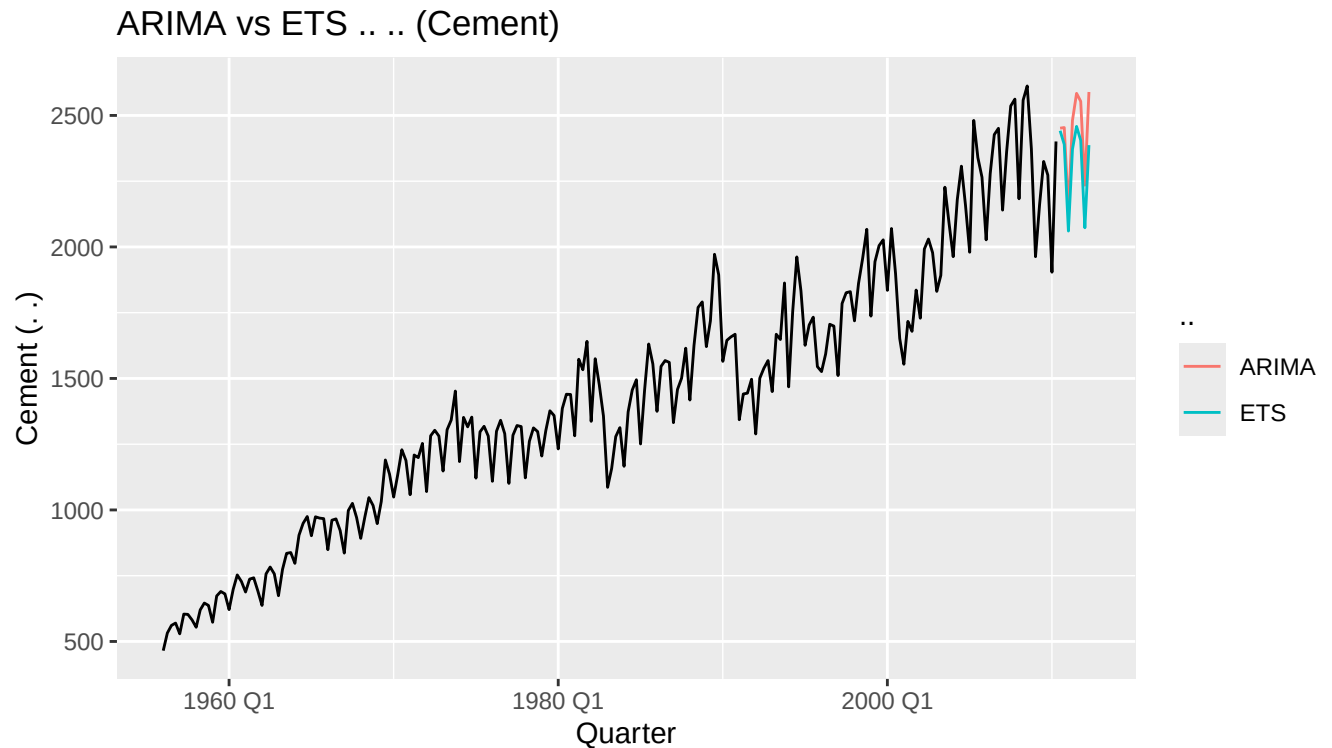
```
best_cement |>
  forecast(h = "2 years") |>
  autoplot(cement) +
  labs(
    title = "      24      ",
    y = "Cement ( )", x = " "
  )
```



(f) ETS와 비교

```
fit_compare <- cement |>
  model(
    ARIMA = ARIMA(box_cox(Cement, lambda_cement)),
    ETS    = ETS(box_cox(Cement, lambda_cement))
  )

fit_compare |>
  forecast(h = "2 years") |>
  autoplot(cement, level = NULL) +
  labs(
    title = "ARIMA vs ETS      (Cement)",
    y = "Cement ( )",
    colour = " "
  )
```



결론: 두 모델의 예측 경로를 비교하여 어느 모델이 더 합리적인지 판단한다.
일반적으로 계절성이 뚜렷한 데이터에서는 두 모델 모두 유사한 성능을 보인다.

Exercise 13

`tourism` 데이터에 ARIMA 모델을 적합시키고 “Snowy Mountains”와 “Melbourne” 예측을 확인하라.

(a) 모든 시계열에 ARIMA 적합

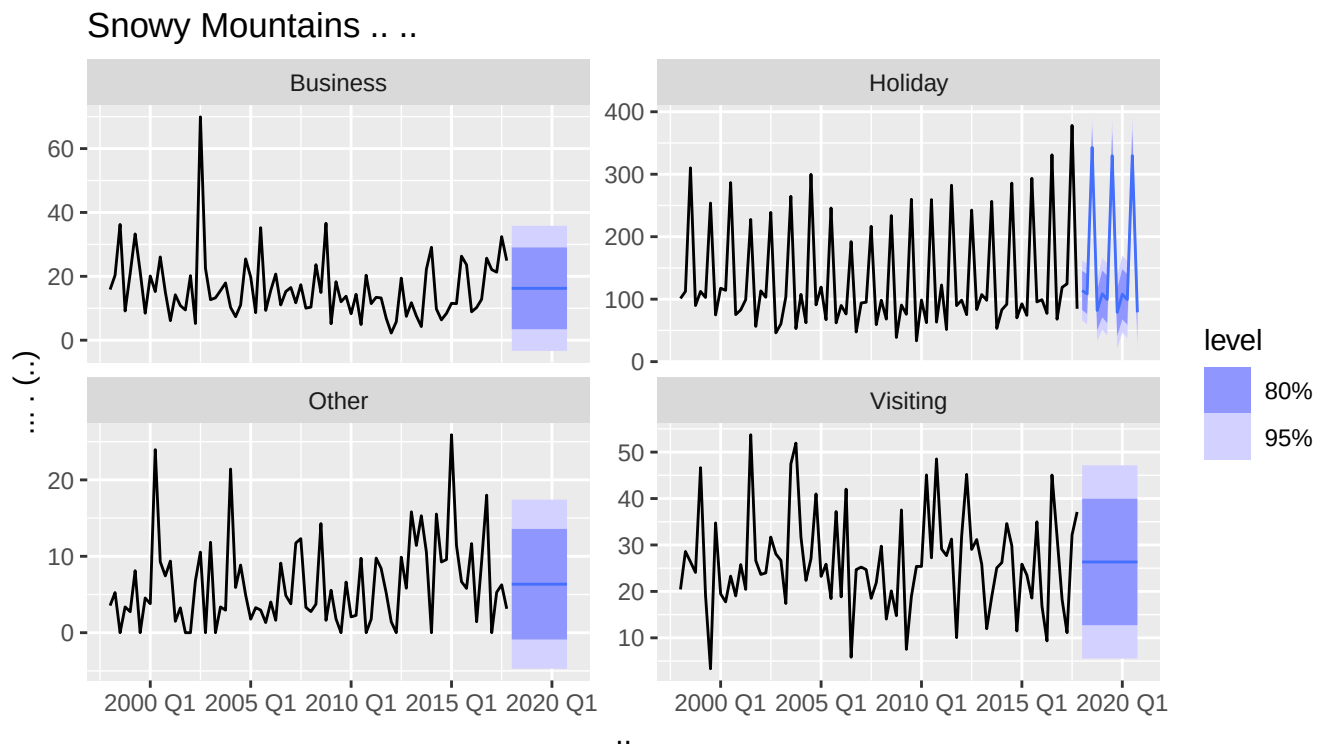
```
fit_tourism <- tourism |>
  model(ARIMA = ARIMA(Trips))
```

(b) 예측 생성

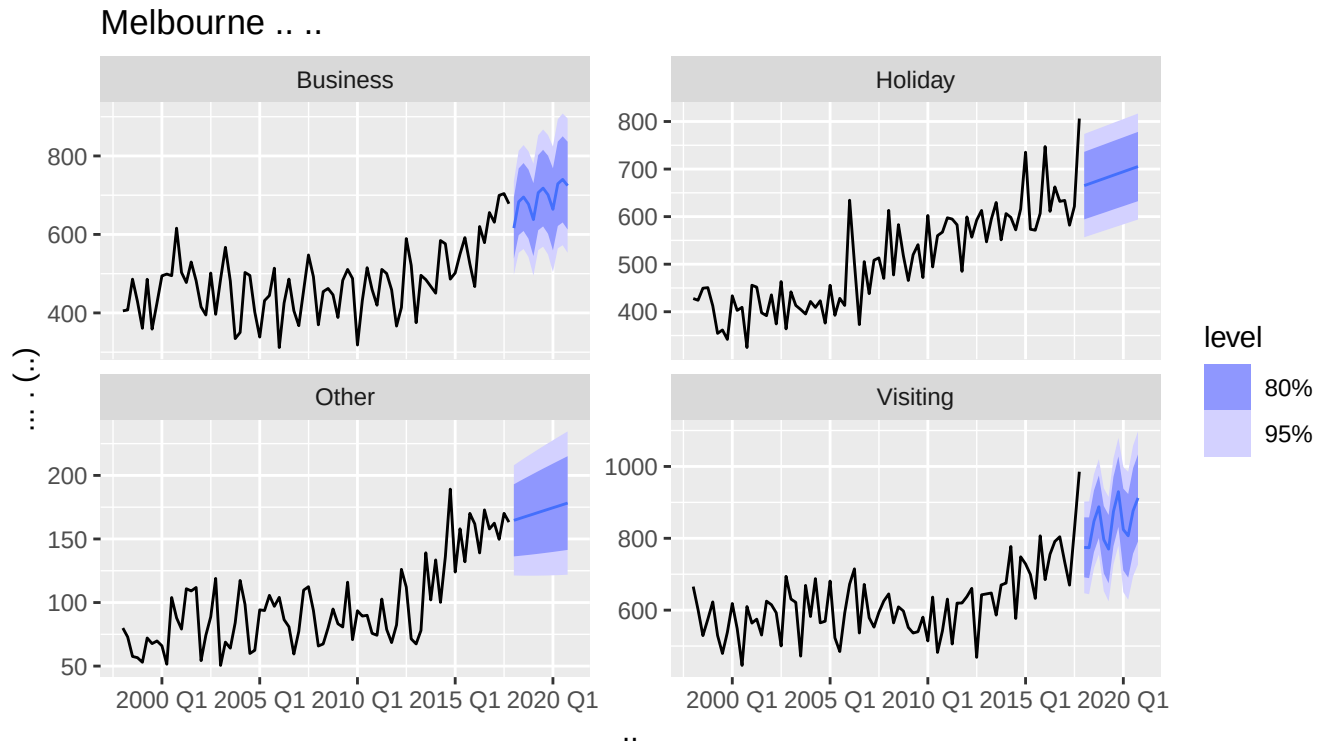
```
fc_tourism <- fit_tourism |>
  forecast(h = "3 years")
```

(c) Snowy Mountains & Melbourne 예측 확인

```
# Snowy Mountains
fc_tourism |>
  filter(Region == "Snowy Mountains") |>
  autoplot(
    tourism |> filter(Region == "Snowy Mountains")
  ) +
  labs(
    title = "Snowy Mountains ",
    y = " ( )", x = " "
  ) +
  facet_wrap(~Purpose, scales = "free_y")
```



```
# Melbourne
fc_tourism |>
  filter(Region == "Melbourne") |>
  autoplot(
    tourism |> filter(Region == "Melbourne")
  ) +
  labs(
    title = "Melbourne ",
    y = " ( )", x = " "
  ) +
  facet_wrap(~Purpose, scales = "free_y")
```



합리성 평가:

Snowy Mountains:

Snowy Mountains는 스키·설경 관광지로 계절성이 극도로 강한 지역이다. Purpose별 특성을 살펴보면:

- Holiday: 겨울(Q3) 집중 패턴이 뚜렷하다. 자동 ARIMA가 계절 차분(D=1)을 포함하면 계절 피크를 잘 포착하는지 확인한다. 만약 예측 구간이 지나치게 넓거나 계절 진폭이 과소/과대 추정되면 모델이 부적합한 것이다.
- Business: 방문객 수가 적고 불규칙하여 단순 ARIMA(0,1,0) 같은 모델이 선택될 수 있다. 이 경우 예측값이 과거 마지막 관측값 주변에서 평탄하게 유지되며 계절성을 전혀 반영하지 못할 수 있으므로, 예측의 활용 가치가 낮다.
- Visiting/Other: 표본이 짧거나 데이터 변동이 크면 자동 ARIMA가 과적합하거나 단순 모델을 선택할 위험이 있다.

Melbourne:

Melbourne은 대도시로 모든 Purpose에서 상대적으로 안정적인 시계열을 보인다.

- Holiday: 완만한 증가 추세 + 약한 계절성이 관찰된다. 자동 ARIMA가 드리프트를 포함한 ARIMA(p,1,q) 계열을 선택하면 증가 추세를 잘 포착한다. 예측이 역사적 성장 경로를 연장하는 형태이면 합리적이다.
- Business: 분기별 패턴이 비교적 규칙적이므로 계절 ARIMA가 적절하다. 예측 구간이 좁고 추세가 유지되면 합리적이다.
- Visiting: 단기 변동이 크지 않아 예측이 안정적으로 나올 가능성이 높다.
- Other: 데이터 볼륨이 작아 불확실성이 크게 나타날 수 있다.

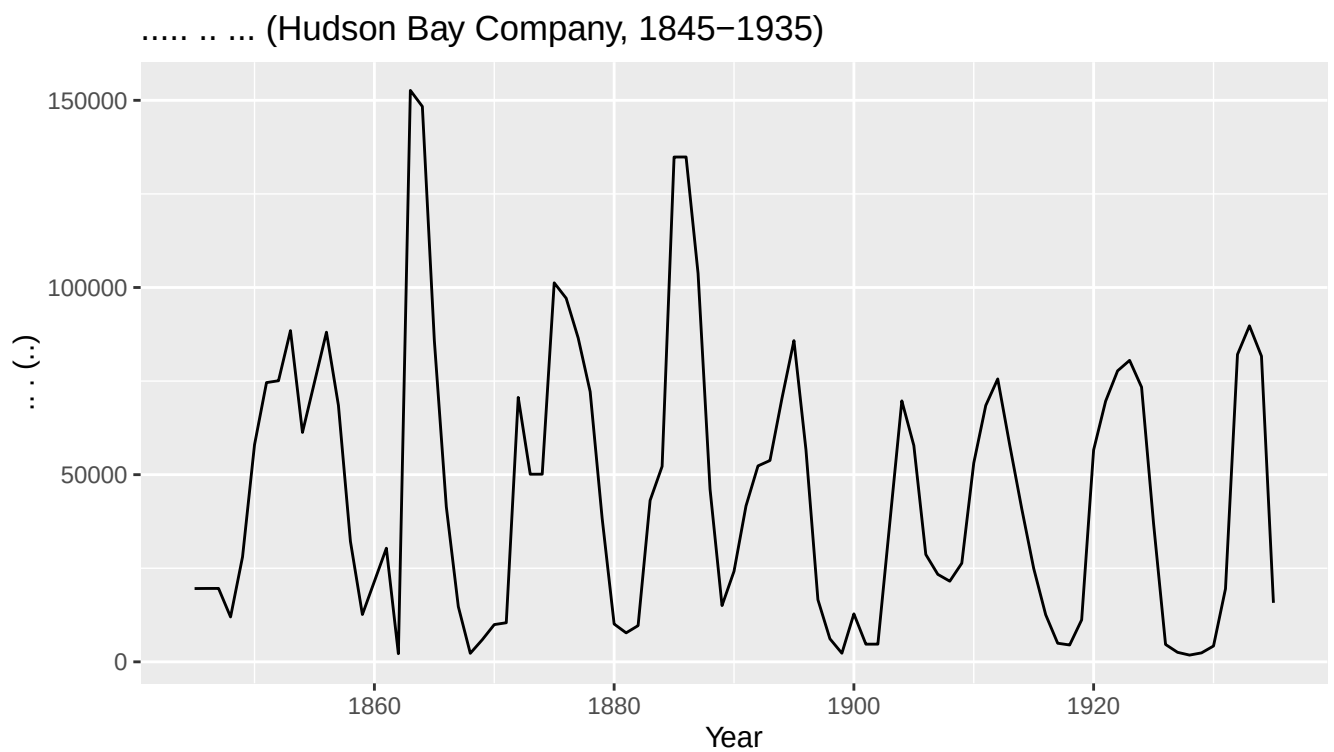
일부 시계열에서 데이터가 불규칙하거나 짧은 경우, 자동 ARIMA가 단순 모델(예: ARIMA(0,1,0))을 선택할 수 있어 예측의 유용성이 제한될 수 있다.

Exercise 15

pelt 데이터의 눈신발토끼(Snowshoe Hare) 모피 거래량 (1845-1935)

(a) 시계열 시각화

```
pelt |>
  autoplot(Hare) +
  labs(
    title = "      (Hudson Bay Company, 1845-1935)",
    y = "      ( )", x = "Year"
  )
```



(b) 모델 식별

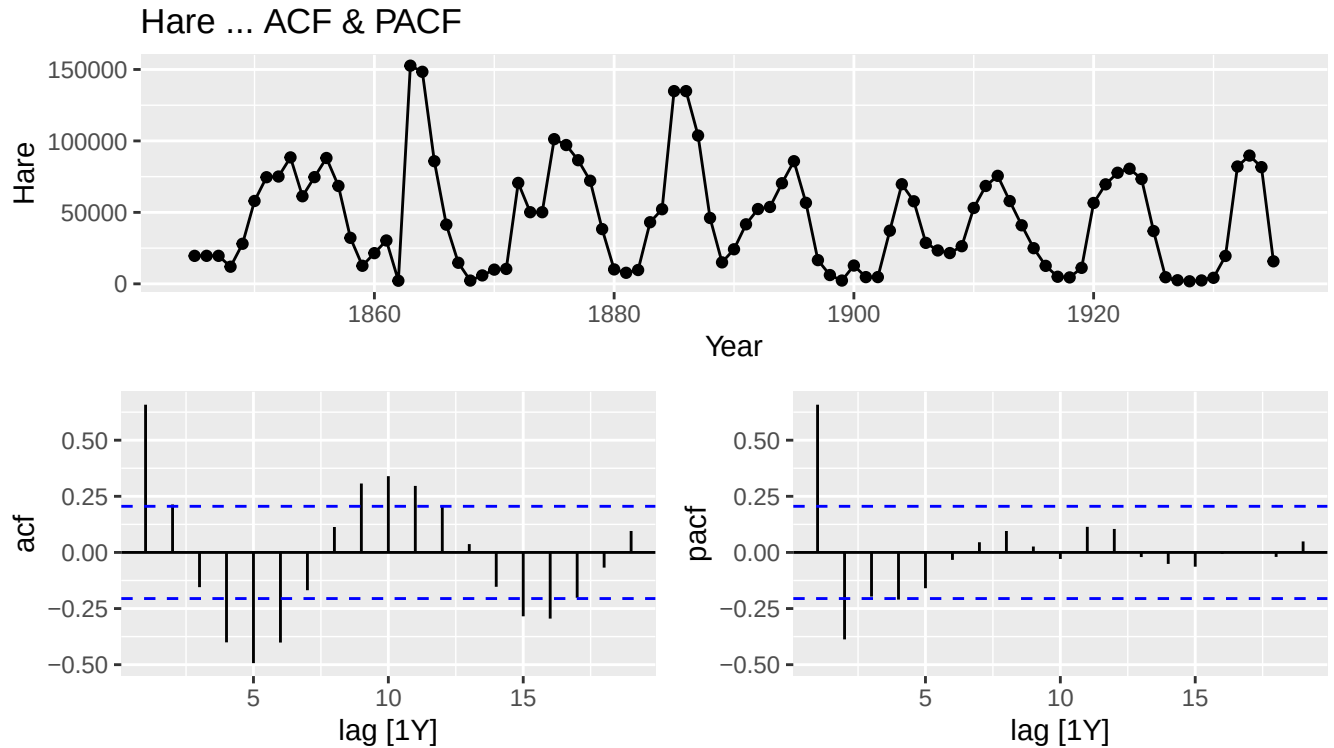
주어진 모델:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + \phi_4 y_{t-4} + \varepsilon_t$$

이 모델은 AR(4) 과정이며, ARIMA 표기로는 ARIMA(4, 0, 0), 즉 $p = 4, d = 0, q = 0$.

(c) ACF/PACF 분석 — 모델의 적절성

```
pelt |>
  gg_tsdisplay(Hare, plot_type = "partial") +
  labs(title = "Hare ACF & PACF")
```



해석:

- ACF: 천천히 감소하는 패턴 (삼각함수적 감쇠) — AR 과정을 시사
- PACF: lag 4 이후 절단(cut-off) — AR(4) 모델이 적합함을 시사

즉, PACF에서 lag 1~4까지 유의하고 lag 5 이후 0에 가까워지므로 AR(4) 모델이 적절하다.

(d) 수작업 예측 (1936–1938, next three years)

마지막 5개 관측값:

연도	1931	1932	1933	1934	1935
Hare	19520	82110	89760	81660	15760

추정 파라미터: $c = 30993$, $\phi_1 = 0.82$, $\phi_2 = -0.29$, $\phi_3 = -0.01$, $\phi_4 = -0.22$

$$\hat{y}_{1936} = 30993 + 0.82(15760) - 0.29(81660) - 0.01(89760) - 0.22(82110)$$

```

c_val <- 30993
phi1 <- 0.82
phi2 <- -0.29
phi3 <- -0.01
phi4 <- -0.22

y1931 <- 19520
y1932 <- 82110
y1933 <- 89760
y1934 <- 81660
y1935 <- 15760

# 1936
y1936 <- c_val + phi1 * y1935 + phi2 * y1934 + phi3 * y1933 + phi4 * y1932
cat("1936 :", round(y1936), "\n")

## 1936 : 1273

# 1937
y1937 <- c_val + phi1 * y1936 + phi2 * y1935 + phi3 * y1934 + phi4 * y1933
cat("1937 :", round(y1937), "\n")

## 1937 : 6903

# 1938
y1938 <- c_val + phi1 * y1937 + phi2 * y1936 + phi3 * y1935 + phi4 * y1934
cat("1938 :", round(y1938), "\n")

## 1938 : 18161

```

문제 원문은 “next three years”이므로 예측 대상은 1936, 1937, 1938 총 3개 연도이다.

(e) R로 모델 적합 및 예측 비교

```

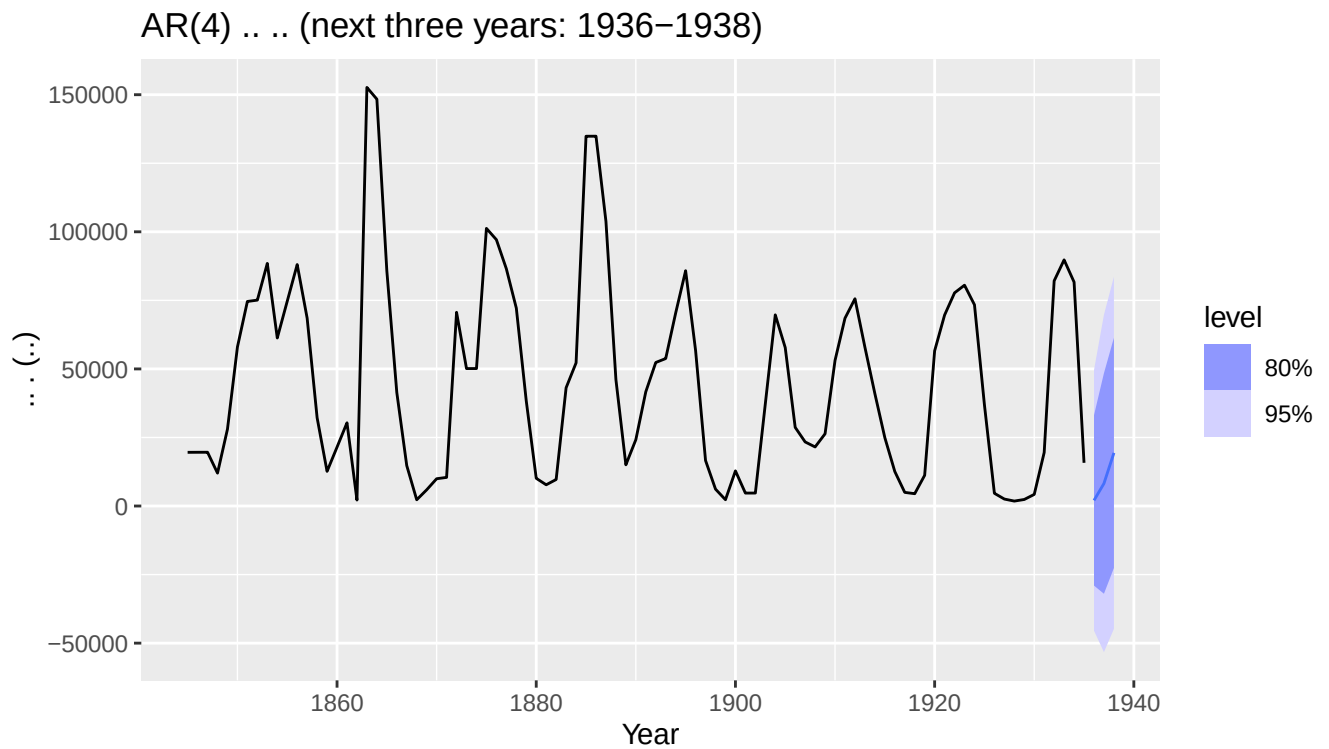
fit_hare <- pelt |>
  model(AR4 = ARIMA(Hare ~ pdq(4, 0, 0)))

report(fit_hare)

## Series: Hare
## Model: ARIMA(4,0,0) w/ mean
##
## Coefficients:
##      ar1      ar2      ar3      ar4  constant
##      0.8219 -0.2891 -0.0057 -0.2165 30993.12
## s.e.  0.1037  0.1367  0.1358  0.1036 2497.78
##
## sigma^2 estimated as 587643879: log likelihood=-1045.91
## AIC=2103.82  AICc=2104.82  BIC=2118.89

```

```
fit_hare |>
  forecast(h = 3) |>
  autoplot(pelt) +
  labs(
    title = "AR(4)      (next three years: 1936-1938)",
    y = "      ( )", x = "Year"
  )
```



수작업 예측과의 차이 원인:

1. R의 `ARIMA()`는 최대우도추정(MLE)으로 파라미터를 추정하므로, 문제에서 제시된 파라미터값과 미묘하게 다를 수 있다.
2. 예측 구간은 추정 불확실성을 반영하여 자동으로 계산된다.
3. 수작업 계산은 점 예측만 가능하지만, R은 예측 분포 전체를 제공한다.